AI Concierge — Semantic Place Intelligence

Architecture Memo and Implementation Blueprint

Author: Architecture working session **Date:** 2026-05-05 **Branch:** `claude/travel-app-architecture-design-yUTIX` **Status:** Architecture / spec / planning only. No code in this document. **Severity classification:** Level 3 (full plumbing analysis + split plan).

---

1. Assumptions

Before designing anything, the assumptions this memo is built on:

1. **Frontier LLMs are available.** We can call Claude Sonnet 4.6 / Opus 4.7 and a Gemini-class model when justified. We are not budget-constrained on intelligence; we are budget-constrained on serial latency and unjustified calls.
2. **Google Places is canonical.** Google Places Text Search, Nearby Search, and Place Details (including the newer review/area summaries on the v1 endpoint) are the source of truth for any addable card. Yelp, Foursquare, Tavily, Brave, Serper, and editorial blogs are enrichment only — never the substrate from which addable cards are minted.
3. **Verified-only addable cards.** A card is addable to an itinerary if and only if it has a stable Google `place_id`, an OPERATIONAL `business_status`, a resolvable Google Maps URI, and survives our trust gates. This is a hard invariant.
4. **Single-replica deploy is the near-term reality.** Railway hosts one process. The in-memory `ContinuationResultPool` (`backend/app/concierge/result_pool.py`) is acceptable for now but must become a Supabase-backed pool by Phase 3 to survive worker recycles and prepare for multi-instance.
5. **Frontend card contract is sticky.** `frontend/src/components/concierge/PlaceRecommendationsView.tsx` already consumes `name`, `cuisine`, `category`, `mapsLink`, `bookingLink`, `sourceUrl`, `supportingDetails.{categoryLabel, metaLine, whyPick}`, `evidence[]`, and `whyPick`. Backend changes must be additive and backwards-compatible. UI redesign is out of scope until Phase 4+.
6. **The user is one well-defined human (and her travel partner).** The wife is the primary user. She speaks naturally, researches deeply, prefers boutique upscale-feel places at honest prices, and wants concierge judgment, not a search box.
7. **Trust is non-negotiable.** Hallucinated views, awards, neighborhoods, opening hours, prices, romance, or "not touristy" claims cause irreversible product damage. We would rather show fewer cards with honest reasons than more cards with fabricated reasons.
8. **Latency is a product feature.** A card list returned in 4 s with one batched honest reason beats a list returned in 90 s with hand-crafted reasons. Latency budget is a first-class invariant, not an afterthought.
9. **Supabase is the persistence layer.** SQL changes must be minimal, additive, rollback-safe, and explained. Schema drift between code and live Supabase has already caused at least one production logging failure (`intent_classifier_version` column missing). Any future schema work assumes a tolerant write path.

10. **The `concierge_request_log` migration `004` exists in the repo** (`backend/db/migrations/004_concierge_request_log.sql:10` declares `intent_classifier_version text`) **but has not been applied to the live Supabase project.** This is the actual root of the `PGRST204` log noise. Either the migration must be applied or the writer must be schema-tolerant. We will recommend both.
11. **There is no vector index today.** Adding one is feasible but not required for Phase 1. We can get most of the way there with the LLM as a query planner over Google Places + structured signals.
12. **Cost ceiling.** A single AI Concierge turn should not exceed roughly 5,000 LLM tokens of intelligence (combined input + output across all calls in the turn) under normal operating conditions. We will not hand-tune below this; we will hand-tune so quality holds at this ceiling.
13. **The product is "luxury for less."** This is a real differentiator and must be encoded as soft preference signals in retrieval + ranking + reasoning, never as a hardcoded "luxury bucket."

---

2. Success criteria

Success is measurable. We commit to the following targets for the AI Concierge feature, set as Phase 1 launch gates and ongoing SLOs.

2.1 Functional success

- **Card-return rate for verified-place asks:** at least **95%** of well-formed natural-language place asks (e.g. "best breweries along the waterfront", "romantic tapas but not too loud") produce **at least one verified addable card.** Anything less is a regression. Text-only fallback for verified-place asks is forbidden when at least one Google-verified candidate exists for any plausible query rewrite.
- **No fake cards ever:** 0 addable cards without a stable Google `place_id` + OPERATIONAL `business_status` + resolvable Google Maps URI. This is a hard zero. CI must enforce it.
- **Reason quality:** Every `whyPick` references at least one concrete attribute of the user's ask (e.g. "tapas / small plates", "near the waterfront", "quiet enough for conversation"). Generic templates are forbidden. A validator rejects reasons containing only superlatives ("a great choice", "highly rated", "perfect spot") with no ask-specific anchor.
- **Follow-up accuracy:** "top 3", "best one", "compare", "which is closest", "less touristy", "more options", "closer to the hotel" each produce the right operation on the prior pool 95% of the time without re-fetching from providers when the pool is sufficient.
- **No category-bucket regressions:** "tapas bar" never returns cocktail bars first. "breweries along the waterfront" never returns generic restaurants first. "speakeasies" never returns hotel lobbies first. CI fixtures lock these in.

2.2 Latency budget

- **p50 end-to-end: < 4.0 s** (Phase 1 target; aspirational < 3.0 s by Phase 3).
- **p95 end-to-end: < 7.5 s.**
- **Hard timeout: 9.0 s** server-side. Beyond this, return a verified-card subset with degraded reasoning rather than blocking.
- **Pool-hit ("more options" with cached candidates):** p95 **< 1.2 s.**

- **Refine-previous ("top 3", "compare", "best one"):** p95 **< 0.8 s** (no provider call, no LLM call required).

2.3 Provider and LLM call budgets per turn

- **Google Places Text Search calls:** ≤ **3** per turn (1 primary + ≤ 2 dynamic refills). Hard cap: 4.
- **Google Place Details calls:** ≤ **8** per turn, only for top-N candidates after rank, executed in **parallel**, with a **400 ms per-call deadline** and best-effort fallback to base text-search fields.
- **Editorial / web evidence calls (Tavily/Brave/Serper):** ≤ **1** per turn, **non-blocking**, used for evidence enrichment only, never for card minting. Capped at 2.0 s; results dropped if late.
- **LLM calls per turn:**
- Frame extraction: **1 call** (Sonnet, ~600 in / ~300 out, ~600 ms target).
- Reason batch: **1 call** (Sonnet, ~3,000 in / ~1,500 out, ~1.6 s target) — covers all final cards in one shot.
- Optional comparator/explanation for follow-up: **1 call** when explicitly requested ("which is more romantic?").
- Hard cap: **3 LLM calls per turn**, all parallel where possible.

2.4 Cache expectations

- **Provider cache hit rate:** ≥ 60% within a single trip session for repeated/similar asks.
- **Place entity cache hit rate:** ≥ 80% across turns within a single trip (since users iterate around the same destination).
- **Result-pool hit rate for "more options":** ≥ 70% on the first follow-up.

2.5 Reason-quality and trust

- **Hallucination audit pass rate: 100%** in nightly fixture run for "claims grounded in provided evidence". Any failure blocks merge.
- **Honesty fallback usage:** when evidence is weak (e.g. "waterfront view" cannot be verified from Google fields or evidence snippets), reasons must say so explicitly ("near the waterfront; verify exact view when booking"), and the validator must accept this phrasing.

2.6 Observability

- 100% of turns emit a structured `concierge.turn` log line containing `request_id`, `trip_id`, frame summary, retrieval plan, candidate counts, rank scores for top-N, evidence used, reason source, latency-per-stage, cache hits, fallback reasons, final card count, validator failures.
- 0% of analytics writes block user response (logger must be try/except + schema-tolerant).
- Schema drift between `backend/db/migrations/*.sql` and live Supabase is detected by a startup check that warns (not crashes) on missing columns.

2.7 UX delight (Phase 3+)

- "What would you personally pick?" produces an opinionated answer with named tradeoff.

- "Make the evening more romantic" rewrites a day plan around a chosen place.
- "Closer to the hotel" reranks the existing pool by Haversine to the resolved hotel address with no provider call.
- "Less touristy" applies a soft penalty for chain/touristy signals from the existing evidence and pool, with no provider call when the pool is sufficient.

---

3. Executive verdict

The current AI Concierge is a keyword-pattern intent router wearing the costume of a semantic search system. Three architectural facts demonstrate this:

1. **The "brain" is `_detect_intent()` at `backend/app/services/concierge.py:672–701`** — thirteen `re.compile(...)` patterns chained as if-elif. "brewery" lives in `_NIGHTLIFE_PAT` (`concierge.py:165–168`) for purely lexical reasons. Whether a user asks for "breweries", "speakeasies", "vinyl bars", "natural wine bars", "omakase counters", "izakayas", "ramen shops at midnight", or "rooftop sunset spots", the system can only see thirteen rooms and tries to stuff every ask into one of them.
2. **The fast path's "subtype" vocabulary is a closed list of 31 cuisines** (`backend/app/services/fast_dynamic_place_search.py:69–101`). It does not contain "brewery", "speakeasy", "natural wine", "tiki bar", "supper club", "dinner theater", "listening bar", "cocktail lounge", or any other long-tail concept that travel customers actually ask for. When the parse cannot find a subtype, the place_type defaults to `"restaurant"` and the category-score gate at `< 0.2` (line ~338) silently rejects every brewery Google returns because the parser told the ranker to expect a restaurant.
3. **There is no LLM in the planning loop at all.** The fast path is a regex parser → fixed Google Text Search → arithmetic ranker → deterministic template reason. The slow path is Tavily article extraction → serial Google verification → per-card LLM reason at ~70 s aggregate. Neither uses an LLM to *think* about the user's ask. The LLM's only role is rationalizing already-chosen cards.

Everything important the system needs to do — interpret the ask, plan retrieval, judge fit, explain choices, handle conversational refinement — is being approximated by string matching. Adding "brewery" to `_NIGHTLIFE_PAT` or to `_SUBTYPE_KEYWORDS` is treating the symptom. The disease is that the architecture cannot accept a long tail of human asks because it was designed around a finite vocabulary.

What is fundamentally wrong

- **Categorical routing as the brain.** The router decides everything downstream. When a category match is wrong or absent, the user gets either silence (text-only response, as in the brewery case) or the wrong vertical (cocktail bars instead of tapas — the 2026-05-04 wife-test failure that motivated PR 3).
- **Two divergent pipelines (fast vs slow) with binary fall-through.** The fast path has trust gates the slow path doesn't, and vice versa. When the fast path returns zero candidates (because the parser misclassifies the ask), the system falls through to a 60–120 s Tavily-based pipeline that mints addable cards from blog posts. That is the wrong substrate. Editorial sources should never be the primary substrate for an addable card; they should enrich a Google-verified entity.
- **Reasons are templated.** `_build_dynamic_why()` outputs sentences like "A stronger tapas/small-plates match than a generic cocktail bar in West Loop." That is not concierge reasoning. It is a filled-in template. Real concierge reasoning would say: "City Mouse on Randolph leans European with a small-plates dinner

format, gets called out for thoughtful natural wine, and reads more dinner-date than nightclub — closer to your tapas brief than the cocktail-first spots above it."

- **Latency is a side-effect of architecture, not a budget.** The 126 s latency on the slow path is because Tavily extraction → serial Google verification → per-card LLM reason runs sequentially with no deadlines. Even the "fast" path runs serially.
- **No conversational memory beyond identity-key dedup.** The system can remember which cards it already showed (good). It cannot remember *why* the wife rejected one and built a preference accordingly (missing).
- **Schema drift between migrations and live Supabase.** Migration `004` exists in repo and declares `intent_classifier_version`, but live Supabase has not had it applied. This produces `PGRST204` errors that the logger does not catch, polluting Railway logs and giving false signal during debugging.
- **No LLM-as-planner.** Modern conversational search systems use the LLM to *plan* retrieval (rewrite the query, decide whether to call providers, decide which providers, decide which fields to fetch). We use the LLM to rationalize cards we already picked.

What should be preserved (the good bones)

- **Identity-key dedup logic** (`backend/app/routes/ai.py:95–172`). The multi-source identity key (`pid:`, `gmaps:`, `name_addr:`) is correct, robust to provider drift, and non-trivial. Keep it.
- **Google Places as the verification gate** (`backend/app/services/google_places.py`). The `OPERATIONAL` business-status gate, the `provider_place_id` requirement, and the `google_maps_uri` requirement are correct and must remain hard invariants.
- **Result pool with TTL** (`backend/app/concierge/result_pool.py`). The continuation pool concept is correct. The implementation is in-memory-only, which is fine for now but must be promoted to Supabase by Phase 3.
- **Typed response contract** (`backend/app/concierge/contracts.py`). The discriminated union (`PlaceRecommendationsResponse | TripAdviceResponse | UnsupportedResponse`) is good. Frontend already keys off `response_type`.
- **Refine-previous card reuse** (`backend/app/concierge/context_resolver.py`). For "top 3", "best one", "compare these", reusing the prior pool with rule-based reranking is correct. No provider call is the right answer.
- **Turn classification primitives** (`backend/app/concierge/context.py`). The shape of `TurnMode` and `RerankRule` is correct. The implementation is regex-based and can be evolved to LLM-augmented.
- **Provider cache** (`backend/app/services/provider_cache.py`). Concept correct; we will reuse it as the provider-tier cache.
- **Frontend card contract** (`frontend/src/components/concierge/PlaceRecommendationsView.tsx`). Card shape is good. We extend it additively.

What should be deprecated

- `_detect_intent()` at `concierge.py:672–701`. Replace with the Frame Extractor. The thirteen patterns become weak features inside the frame, not the brain.

- `_SUBTYPE_KEYWORDS` / `_VIBE_KEYWORDS` / `_CONSTRAINT_KEYWORDS` **as the only ontology** at `fast_dynamic_place_search.py:69–132`. Demote to "weak hints inside the frame", augmented by LLM-extracted features.
- `LiveResearchService.fetch()` **as a substrate for addable cards** at `backend/app/services/live_research.py`. Tavily content remains as **evidence** (snippets supporting reasons), not as the source of addable entities. The fundamental architectural shift: Tavily article → addable card is wrong. Verified Google place → optional Tavily snippet supporting a claim is right.
- **Per-card LLM reason generation** (any path that calls the reason LLM N times for N cards). Replace with a single batched call.
- **The "fast vs slow path" binary fall-through.** Replace with one pipeline where the LLM-planned retrieval is always first, with optional evidence enrichment in parallel and best-effort.
- `_NIGHTLIFE_PAT` **containing "brewery"** at `concierge.py:165–168`. Removed entirely once the Frame Extractor exists. Do not add more keywords; remove the file's role.

Why adding more category buckets is the wrong durable solution

Three reasons:

- **The vocabulary is unbounded.** Every category we add (brewery, speakeasy, listening bar, omakase, izakaya, cevicheria, taqueria, gastropub) creates an asymmetric maintenance liability. The next ask the wife makes will be the one we forgot. Each new keyword also creates new edge cases ("brewery tour", "brewery with food", "brewery with patio") that the regex cannot cleanly compose.
- **It cannot encode soft constraints.** "Best breweries along the waterfront, not too touristy, ideally walkable from our hotel, with patio seating" has six concept slots. A bucket router can only match the first noun. A frame-based system encodes all six.
- **It does not scale to "luxury for less" or comparison reasoning.** "Worth the splurge?" or "what would you personally pick?" cannot be expressed as a category. They are reasoning operations on an entity set. We need an architecture that treats the user's ask as a flexible structure, not a vertical to dispatch into.

Why this feature deserves a deeper architecture

The AI Concierge is the flagship product surface. It is the difference between "another itinerary builder" and "a concierge inside your trip." The wife's asks are not unusual; they are exactly what travel customers actually say. The architecture must be capable of accepting them as-is, planning intelligent retrieval, verifying entities, fusing evidence, ranking semantically, and reasoning honestly. None of that is overengineering. All of that is the minimum bar to compete with what Google Maps Ask Maps, Tripadvisor AI Trip Builder, and Perplexity travel search will keep getting better at.

---

4. Competitive reverse engineering

For each comparable system, the questions are: what is the likely architecture, what should we copy, what should we reject, and what does our addable-card-inside-the-trip context let us do better?

4.1 Google Maps "Ask Maps" / Gemini conversational maps

Likely architecture. A Gemini model receives the user's natural-language ask plus contextual signals (location, current map view, time of day). The model performs LLM-driven query planning, calls Google Places under the hood (Text Search, Nearby, Place Details, the `placeSummary` and `reviewsSummary` v1 endpoints), uses Google's review-summarization to surface "what reviewers say", and returns a chat answer with a small carousel of pinned places. The retrieval planner is the LLM; the ranker is a fusion of Google's existing relevance signals plus query-fit features the LLM derives.

Strengths. Massive entity graph, world-class verification, review summaries grounded in real reviews, area summaries, photo coverage. Latency is excellent because the entire stack is co-located.

Weaknesses. It is a generic "ask the map" surface, not a concierge-inside-a-trip. It does not know about your hotel, your booked dinners, your itinerary, or the rest of your day. It lacks a persistent travel-context layer. It is also conservative on opinions — it will not say "skip Wicker Park, go to Logan Square instead, the wine list is better and the room is quieter." It hedges.

Copy. LLM-as-planner. Use Place Details review/area summaries as evidence. Multi-step retrieval (text search → details → photos in parallel).

Reject. Hedged unopinionated reasons. The carousel-as-final-result UX without addability into a structured trip. Chat-as-the-only-surface — we have first-class itinerary cards.

Where we can win. We have the trip context (destination, dates, days, hotel, prior selected cards, prior rejected cards, the rest of the user's plan). We can answer "closer to the hotel" or "after dinner" with structural certainty. We can render addable cards that materialize into the user's actual itinerary. We can be opinionated where Google Maps is institutionally hedged.

4.2 Google Places API and Place AI summaries

What it actually is. A REST API. Text Search v1, Nearby Search v1, Place Details v1 with field masks. Newer endpoints expose `reviewSummary` and `areaSummary` content (limited rollout, gated by region and account). Photos endpoint, contact info, business status, opening hours, types, ratings, user rating count, price level (where set), website, maps URI.

Strengths. Authoritative business graph. Stable IDs. Operational status. Maps URIs. Excellent recall and precision for "what is here and what is it called."

Weaknesses. It is not a concept understanding system. It does not know "speakeasy" or "listening bar" with high confidence; it knows `bar`, `night_club`, `restaurant`. The `types` taxonomy is coarse. Subtype resolution requires combining Text Search query (which is generative and free-form) with type hints and review evidence.

How we use it. As the verification spine. Every addable card has a Google `place_id`. Text Search is the primary retrieval call (we send the literal user ask + destination context). Place Details is called *after rank*, in parallel, only for the top-N candidates we plan to show. `reviewSummary` (when available) is a primary evidence source. Photos URLs are populated only for the final top-N.

What we will not do. Use the `types` field as the primary classifier. Use the `types` field as a hard gate. Both are too coarse. Types are a weak feature inside the ranker, not the gate.

4.3 Tripadvisor AI Trip Builder

Likely architecture. A travel-domain LLM with access to Tripadvisor's review corpus and entity graph. A trip-builder agent that generates day plans from category-aware retrieval over their POI database. Heavy reliance on aggregated review sentiment, "Travelers' Choice" awards, and category breakdowns.

Strengths. Massive review corpus, structured user-generated category data, opinionated awards. Trip-level structure (days, destinations).

Weaknesses. Ranking is dominated by popularity / review volume → tourist-trap bias. The "best Italian in Rome" answer will be the most-reviewed Italian, which is often the most-touristed Italian, which is often not the best. Conversational refinement is shallow. Cards are not deeply addable into a personal itinerary that the user owns.

Copy. Day-aware planning. Category breakdowns as a soft feature. Review-based evidence.

Reject. Popularity-weighted ranking as the dominant signal. Tourist-trap bias. Awards-as-truth.

Where we win. Our "less touristy" preference can be a real ranking penalty, not a soft filter. We can use the Google review corpus combined with Tavily editorial enrichment to capture "locals love this" signals that Tripadvisor's surface masks. We can de-bias the popularity signal explicitly.

4.4 Expedia Romie

Likely architecture. Trip-level conversational assistant integrated with Expedia's booking inventory. Strong on booking flows, less on POI discovery. Likely uses retrieval over inventory + LLM for trip-shaping language.

Strengths. Booking integration. Trip-level conversational continuity.

Weaknesses. Inventory bias — recommendations skew to what Expedia can monetize. POI discovery is shallow compared to Google Maps.

Copy. Trip-level conversational continuity. Conversational refinement of an existing plan.

Reject. Inventory-biased recommendations. We are not selling rooms; we are giving advice.

Where we win. No inventory bias. Pure advice optimized for the user's experience, not for our affiliate margin.

4.5 Perplexity travel / generic Perplexity search

Likely architecture. Multi-step planner. Decomposes the question into search queries. Retrieves from web (Bing/Google index + arxiv + reddit + others). Re-ranks. Calls an LLM to synthesize a grounded answer with citations. For travel, Perplexity adds entity recognition for places and renders a small map with pinned results.

Strengths. Excellent generative query planning. Citations. Up-to-date information through fresh web retrieval. Good on long-tail queries.

Weaknesses. Web sources are noisy; many results are SEO content farms ("17 Best Brunch Spots in Chicago Updated 2025") that tend to converge on the same overhyped venues. No verified entity layer — the addable place is whatever the web article says, not a Google-verified entity. The chat surface is not an itinerary.

Copy. LLM-driven query decomposition and rewriting. Citations as evidence anchors. The "show your work" UX where the user can see what was searched.

Reject. Web articles as the substrate for addable entities. SEO content farm convergence. Lack of trip context.

Where we win. Verified place entities. Editorial content used only as evidence supporting Google-verified entities. Trip-context-aware retrieval. Cards that go into the user's itinerary, not links that go nowhere.

4.6 Generic ChatGPT travel planning

Likely architecture. Frontier LLM with browsing tool use. The model decides whether to browse, generates queries, summarizes results, returns a long-form recommendation.

Strengths. Conversational fluency. Long-form synthesis. Can take ambiguous asks and produce something useful.

Weaknesses. Hallucinates. Will confidently invent a "Lakefront Brewery" that does not exist or got the address wrong, name the wrong neighborhood, or claim Michelin status that was rescinded two years ago. No verified entity layer, no addable cards into a real itinerary. No latency budget. No structured day plan.

Copy. Conversational fluency. Tone. Willingness to be opinionated.

Reject. Unverified entities. No trust gates. No structured cards.

Where we win. Trust. Verified addability. Trip context. Structured day planning. Latency budgeting.

4.7 OTA search/filter systems (Booking, Hotels.com, Kayak)

Likely architecture. Faceted filter UI over a structured inventory. Filters: price, rating, distance, amenities, brand. No conversational layer. Sort by relevance/price/rating.

Strengths. Precision when the user knows the filters they want. Fast.

Weaknesses. Cannot answer "best breweries along the waterfront not too touristy" because none of those are filterable facets. The user must translate their ask into filters they may not have.

Copy. Hard constraint satisfaction logic. Geography filtering. The respect for "the user said proximity, give them proximity."

Reject. Filter UIs as the only surface. Lack of natural-language layer.

Where we win. Natural-language ask → frame → constraints. We do the filter translation for the user.

4.8 Yelp / Foursquare local discovery

Likely architecture. Category trees + review-driven ranking. Faceted filters. Personalization via past saves and ratings.

Strengths. Rich category hierarchy. Tip/review depth. Local signals (especially Foursquare's check-in graph historically, although their API has narrowed).

Weaknesses. Category trees are rigid. The "is this place actually romantic?" question is approximated by review keywords with significant noise. Bias toward urban-American-centric categorization.

Copy. Category subtypes as features. Tips/review snippets as evidence.

Reject. Category trees as the brain. Review-volume-weighted ranking.

Where we win. Frame-based reasoning over multiple evidence sources. We do not depend on Yelp's category tree being right.

4.9 Modern RAG patterns (general, applied to local recommendations)

What is now standard. Hybrid retrieval (BM25 + dense embeddings). Query rewriting (HyDE, generated query expansion). Reranking with cross-encoders or LLM rerankers. Citation-grounded synthesis with provenance tracking. Tool use for structured knowledge bases.

For local recommendations specifically. Add geographic constraints as hard filters. Use entity disambiguation (which "Au Cheval"? the Chicago original or the New York spinoff). Use freshness (places open/closed). Use multi-source evidence fusion.

Copy. Generated query expansion (LLM-as-planner). Reranking after retrieval. Multi-source evidence with provenance. Citation grounding for reasons.

Reject. Pure dense retrieval without an authoritative entity layer for local — embeddings cannot tell us if a place is currently open. We need Google as the authoritative entity layer.

Where we win. We combine Google's authoritative entity layer with LLM-driven retrieval planning and multi-source evidence. Most pure-RAG implementations omit the authoritative entity layer.

4.10 LLM-as-planner vs deterministic verification

The right pattern is well-established: LLM plans, deterministic systems verify. The LLM extracts the user's experience frame and generates retrieval queries; the system runs them deterministically; the system verifies entities deterministically against an authoritative source (Google); the LLM does the final reasoning grounded in fetched evidence; deterministic validators reject hallucinated reasoning. This is the architecture this memo proposes.

4.11 Multi-stage ranking systems

Industry pattern. Stage 1: cheap recall (~hundreds of candidates). Stage 2: feature-based ranker (~tens). Stage 3: expensive cross-encoder or LLM reranker (~final 5–10). Diversity reranking last.

For us: Stage 1 = Google Text Search candidate fanout. Stage 2 = deterministic feature ranker (subtype fit, geo fit, vibe fit, popularity, freshness). Stage 3 = LLM batched judgment on top-N for final ordering and reason. Stage 4 = diversity / dedup pass.

4.12 Agentic UX for conversational refinement

Industry pattern. Persistent chat with explicit "I understood you to mean X — is that right?" disambiguation, optional refinement chips ("more upscale", "closer to me", "open late"), and ability to inspect the search trace.

Copy. Persistent context. Subtle refinement chips driven by frame deltas. The pool concept (we already have it).

Reject. Heavy disambiguation prompts. The user said "best breweries along the waterfront" — answer it; do not interrogate.

---

5. Product vision

The north-star AI Concierge feels like one specific person: a smart, well-traveled friend who lives in this city, knows you well enough to know your wife likes natural wine and quiet rooms, has been everywhere, has an opinion, and respects your time.

5.1 How chat should behave

- **Receptive.** It accepts long, soft, vague human asks without "please be more specific" pushback.
- **Confident.** It returns its best answer first, with a one-line reason that makes the choice feel inevitable.
- **Honest.** When evidence is weak, it says so plainly: "I cannot verify the patio is heated in May, but the room reads warm and the back has windows facing the river."
- **Opinionated.** When asked "which would you pick?", it picks and explains.

- **Sticky on context.** It remembers the trip, the hotel, what was added, what was rejected. It uses that to refine without being asked.
- **Crisp.** No "Here are some great options for you to consider!" preamble. Lead with the recommendation.

5.2 How cards should behave

- **Verified, addable, fast.** Every card is a Google-verified place with an addable button.
- **Ask-specific reason.** Every `whyPick` is anchored to *this user's ask*, not to the place's generic identity.
- **Evidence transparent.** Optional "source" badges link to the review / article / Google area summary that supports the reason.
- **Comparable.** Tapping any card opens an expanded view; selecting two opens compare mode.
- **Honest tradeoffs.** Cards optionally expose "good fit, weaker waterfront evidence" or "best overall but not closest to hotel" labels.

5.3 How reasons should read

The bar: a reason should be one or two sentences that sound like a friend, are anchored in the ask's actual concepts, and reference specific evidence. Examples (illustrative — not template):

Before / current (`fast_dynamic_place_search._build_dynamic_why`): > "A stronger tapas/small-plates match than a generic cocktail bar in West Loop."

After / target: > "Cira at Hoxton leans Mediterranean small-plates with a quieter dinner room than the Randolph crawl, and reviewers consistently call out the romance over the volume — a closer match to your tapas brief than the cocktail-first rooms above it."

Before / current (a brewery example, hypothetical): > "A great brewery option in the area." (clearly bad)

After / target: > "Goose Island Fulton Taproom is the only brewery within walking distance of the river-walk segment your hotel sits on; it is brewery-first, not a brewpub-with-views, so think 'taproom-on-the-water', not 'brewery with skyline patio'."

Before / current (waterfront-claim example): > "Located near the waterfront."

After / target: > "Two blocks from Riverwalk between Wabash and State; the room itself does not face the water but the post-dinner walk takes you straight to the river."

The new reasons trade fluency for honesty and specificity. They acknowledge what the place is and what it is not. They use verified geography ("two blocks from Riverwalk between Wabash and State") rather than hedged poetic language ("near the waterfront").

5.4 How follow-ups should work

- **"top 3"** → uses the existing pool. No provider call. Sub-second response. Shows the top 3 by current rank with a subtle "showing top 3" header.
- **"more options"** → pool fast path. Returns next batch. If pool empty or insufficient, runs *one* targeted refill query (different geographic seed or different subtype fan-out), respecting prior identity keys.
- **"which is more romantic?"** → comparator LLM call over the pool's evidence. Returns a one-paragraph judgment with a recommendation.

- **"closer to the hotel"** → reranks pool by Haversine to resolved hotel. No provider call.
- **"less touristy"** → applies tourist-mass penalty (review-count z-score, chain-detection signal, "tourist trap" review-keyword density) to the existing pool. Reranks. No provider call unless the pool top-3 still all fail the threshold, in which case one refill query with a "local-favorite" framing is allowed.
- **"I do not want a chain"** → applies chain detector (deterministic check on `name` against a small known-chain list, augmented by a soft LLM signal); rerank pool. No provider call unless pool collapses.

5.5 How trip context should influence answers

- **Hotel proximity** is a soft factor on every retrieval and a hard factor when the user invokes it ("near the hotel").
- **Day timing** influences ranking: lunch vs dinner vs late-night queries weight opening hours and time-of-day evidence differently.
- **Existing itinerary** influences diversity: if the user already has Italian dinner Wednesday, "more dinner ideas" should not return three more Italian places.
- **Selected vs rejected cards** become preference signals: rejected ■ negative weight on attributes shared with the rejection; selected ■ positive weight.

5.6 How personalization evolves

Personalization is layered:

- **Per-trip:** every accepted/rejected card updates per-trip soft preferences.
- **Per-user:** persistent across trips. Slowly accumulates "user dislikes loud rooms", "user values walkability", "user has cuisine breadth".
- **Per-couple:** at the trip level, we may capture "wife likes natural wine, husband likes mezcal" as soft signals influencing reasons.

Personalization is never a hard filter. It biases ranking and reasoning, never excludes.

5.7 How "luxury for less" is encoded

Three signals, combined:

- **Boutique-feel signal:** evidence keywords like "intimate", "craft", "thoughtful", "destination room", "elevated" drawn from real reviews. Distinct from "elegant" or "luxe" markers that often signal hotel-restaurant chain feel.
- **Value signal:** price level + review sentiment on value. A `price_level=2` with reviews emphasizing the experience is favored over `price_level=4` with reviews emphasizing the bill.
- **Anti-touristy signal:** review-count z-score within destination + chain detection + tourist-keyword density.

The combination produces a "luxury-for-less" score that biases ranking when relevant signals are present in the user's ask ("upscale but not splurgy", "boutique", "fancy dinner without breaking the bank").

5.8 Concrete before/after examples

Ask	Before	After
"best breweries along the waterfront"	text-only response, no cards (current production failure)	4 verified Chicago breweries with explicit "this one is on the riverwalk", "this one is two blocks back but the rooftop overlooks the lake", "this one is brewery-first not brewpub" reasons
"romantic tapas but not too loud"	INTENT_NIGHTLIFE → cocktail bars (the 2026-05-04 wife-test failure that motivated PR 3)	tapas-specific small-plates rooms with quiet/dinner-date evidence; cocktail-only bars demoted
"more options"	provider call + dedup; sometimes returns 1 card	pool fast path; 5 fresh cards in <1.2 s
"which is more romantic?"	not currently supported	comparator LLM call returning "Cira reads more romantic than Bocadillo because [evidence]; pick Cira if the ask is the wine list, Bocadillo if it is the patio"
"closer to the hotel"	not consistently supported	rerank existing pool by Haversine to hotel; no provider call
"luxury for less"	no specific behavior	ranker shifts toward boutique-feel signals; reasoner explains the value tradeoff

---

6. Root cause model

The brewery query failed for compounded reasons. Tracing from the top:

6.1 The classification step succeeded — and that hid the problem

Per the user's Railway log:

```
turn_mode=new_search rerank_rule=none card_pool_size=0 has_prior_cards=False
provider_call_expected_for_future_mode=True ...
intent_classifier_version=router_v2.1 intent_confidence=1.0000
response_type=place_recommendations latency_ms=6032 sources_used=[]
```

Router v2 correctly classified the ask as `place_recommendations`. Confidence was 1.0. The high-level routing was right. The system *thought* it was answering a place ask. So the "no cards" outcome is not a routing failure at the typed-contract level. It is a *retrieval and verification failure downstream of routing*.

6.2 Where the brewery ask actually gets stuck

Tracing the call path for "best breweries along the waterfront" with destination = Chicago:

1. `_detect_intent("best breweries along the waterfront")` at `concierge.py:672–701`. The pattern `_NIGHTLIFE_PAT` (`concierge.py:165–168`) contains the literal substring `brewery`. The intent is set to `INTENT_NIGHTLIFE`.

1. `_fetch_live_research()` at `concierge.py:595–670`. With `concierge_fast_dynamic_place_search_v1_enabled=True` and intent in `_FAST_DYNAMIC_INTENTS`

(which includes `INTENT_NIGHTLIFE`), the call delegates to `fast_dynamic_place_search.search()`.

1. `parse_place_query("best breweries along the waterfront", "Chicago")` at `fast_dynamic_place_search.py:223–301`. The parser:

- Walks `_SUBTYPE_KEYWORDS` (`fast_dynamic_place_search.py:69–101`). **"brewery" is not in the list.** No cuisine match.
- Walks `_VIBE_KEYWORDS`. No match.
- Walks `_CONSTRAINT_KEYWORDS` (`fast_dynamic_place_search.py:117–132`). Matches `waterfront` as a constraint.
- The `place_type` defaults: no `bar` keyword in the prompt, no cuisine, no explicit restaurant signal → falls into the `restaurant` default (line ~282 of the parser).
- `canonical_query = "best breweries along the waterfront"` (preserved literally).
- `search_query = "best breweries along the waterfront Chicago"`.

1. **Google Text Search** is called with `search_query = "best breweries along the waterfront Chicago"`. Google returns 15 candidates. They include real breweries (Lagunitas Taproom, Goose Island, Revolution Brewing, etc.) tagged as `bar` / `night_club` / `brewery` / `food` types.

1. `_category_score(place, parsed)` at `fast_dynamic_place_search.py:338–349`. The parser told the ranker to expect `place_type="restaurant"`. None of the brewery candidates score as restaurants. They score as bars. The `< 0.2` cutoff rejects them.

1. **Result:** zero verified candidates pass the gate. `final_unique_count = 0`.

1. **Fall-through to slow path** (`live_research.fetch()`). The slow path calls Tavily for "best breweries along the waterfront Chicago". Tavily returns articles. The serial verification loop attempts to verify the article-extracted brewery names against Google. In practice for this query, the slow path returns sources_used=[] in the user's log, meaning Tavily either timed out, failed, or returned nothing the verification loop could lift.

1. **Final response:** `card_pool_size=0`, no addable cards, text-only fallback per the typed-contract spec, latency `6032 ms` — six seconds spent producing nothing.

6.3 Why `sources_used=[]` matters

`sources_used=[]` in the persisted log is the smoking gun: the slow path's provider extraction returned zero usable sources. Combined with `card_pool_size=0`, this means *no candidate from any path made it through the verification gate*. The system silently degraded to text-only without telling the user that the verification gate rejected everything. This is dangerous because the user has no way to distinguish "no breweries on the waterfront" (a true negative) from "the system misclassified the ask and rejected real candidates" (the actual case).

6.4 Why `card_pool_size=0` matters

`card_pool_size=0` means the trip context window also had no prior cards to fall back on. This is `has_prior_cards=False`, an early-trip ask. There was no pool to recover from. Combined with the fast-path empty result + slow-path empty result, the system had nothing left to return.

6.5 Why text-only fallback for a verified-place ask is dangerous

The user asked for places. Returning text instead of cards violates the implicit contract. The wife sees "here are some great breweries to check out: …" with names but no addable cards, no map links, no addability into her itinerary. This is functionally a search engine answer, not a concierge answer. It also creates a UX dead-end — the wife now has to manually paste names into Google Maps to verify them. The whole point of the product is gone.

The architectural fix: **never return text-only for a verified-place ask if any plausible query rewrite produces verified candidates.** If the system is unsure whether to call providers, it must err on the side of trying multiple rewrites, not on degrading to text.

6.6 Why logging schema drift hurts debugging

The Railway log shows:

```
ERROR:app.concierge.logging:concierge.request_log.persist_failed
postgrest.exceptions.APIError: {'message': "Could not find the
'intent_classifier_version' column of 'concierge_request_log' in the
schema cache", 'code': 'PGRST204', ...}
```

This is migration drift. Migration `004_concierge_request_log.sql:10` declares the column. The live Supabase project has not had `004` applied (or its schema cache is stale). Two consequences:

- **Every concierge turn logs an ERROR**, polluting Railway and creating false alarm fatigue. When a real bug appears, this noise hides it.
- **Analytics is incomplete.** The intent_classifier_version field is precisely what we need to differentiate router_v1 from router_v2 from a future router_v3. Losing it now means we cannot retrospectively measure router improvements.

The architectural fix: the analytics writer must be **schema-tolerant** — catch `PGRST204` and similar, log a single warning at startup, drop the missing field, retry without it. *And* `004` should be applied to live Supabase. Both, not one.

6.7 The deeper architectural flaw this reveals

The brewery failure is not a brewery bug. It is an architecture in which:

- The ask is parsed into a closed vocabulary that does not contain the user's actual concept.
- The retrieval planner is a string template, not an LLM that can reason over the ask.
- The verification gate rejects candidates based on a misclassified intent, not based on whether the candidates are real, operational, and plausibly relevant.
- The fall-through path mints addable cards from the wrong substrate (web articles).
- The system silently degrades to text without recovery attempts.
- The observability layer is broken in ways that mask further bugs.

Fix the architecture, the brewery test passes incidentally — and so do the next thirty asks the wife will make that we have not anticipated.

---

7. Proposed architecture

The new architecture replaces "category routing" with **Semantic Place Intelligence**: a small set of well-bounded modules that pass typed objects to one another, with the LLM doing planning and reasoning, deterministic systems doing verification and ranking, and clear deadlines on every step.

7.1 High-level flow

```
User prompt + trip context + prior pool
        |
        v
[A] Conversation/Turn Interpreter           — fast classifier; route to one of the 6 turn modes
        |
        v
[B] Experience Frame Extractor              — LLM #1, 1 call, ~600ms, produces an ExperienceFrame
        |
        v
[C] Retrieval Planner                       — deterministic planner over the frame; produces a RetrievalPla
        |
        v
[D] Provider Query Generator + Executor     — runs Google Places calls in parallel under a deadline
        |
        v
[E] Verified Place Entity Layer             — canonicalize, dedup, gate for OPERATIONAL + place_id
        |
        v          (parallel fan-out from this point)
[F] Evidence Collector / Fuser              — Google reviewSummary, areaSummary, optional Tavily snippet, g
        |
        v
[G] Semantic Fit Ranker                     — feature-based ranker; produces top-N
        |
        v
[H] Reasoning Engine                        — LLM #2, 1 batched call over top-N, produces whyPick + tradeof
        |
        v
[I] Result Pool / Cache writer              — store typed result + identity keys
        |
        v
[L] Safety/Trust Gate (final pass)          — validator; reject reasons that fail evidence-grounding
        |
        v
[K] Observability emitter                   — structured log with everything
        |
        v
[M] Frontend response (typed contract)
```

For follow-ups, the path is shorter: **[A]** → **[J] Follow-up Engine** (which may use [G] [H] over the existing pool) → **[L]** → **[K]** → **[M]**. Most follow-ups skip provider calls entirely.

7.2 Module-by-module spec

A. Conversation / Turn Interpreter

- **Purpose.** Decide the operation type before doing anything expensive: `new_search`, `more_options`, `refine_previous`, `compare`, `anchor_new`, `reset`. Output a `TurnMode` and `RerankRule`.
- **Inputs.** User prompt, prior cards (count + identity keys), prior prompts (last 3), feature flags.
- **Outputs.** `TurnMode`, `RerankRule`, `confidence`, optional `reset_reason`, optional `selected_card_anchor_id` (if "near the first one").
- **Deterministic vs LLM.** Hybrid. Cheap regex first (current `context.classify_turn`). When confidence is below a threshold OR the prompt is long/ambiguous, escalate to a small LLM call (Haiku-class) that returns the same typed `TurnMode`.
- **Failure behavior.** Default to `new_search`. Never block. The risk of mis-routing to `new_search` (extra provider call) is much lower than mis-routing to `refine_previous` (returning stale cards).
- **Latency budget.** 50–200 ms.
- **Tests.** Unit tests covering: "top 3", "best one", "compare", "more options", "near the first one", "reset", "near the hotel", "after dinner", "for tomorrow", "I do not want a chain", and combinations.
- **Existing code reused.** `backend/app/concierge/context.py` `classify_turn`, `is_more_options_continuation`, `derive_*_hint`. The signature is preserved; the body becomes hybrid-LLM-augmented.
- **Existing code deprecated.** None.

B. Experience Frame Extractor

- **Purpose.** Turn the user's natural-language ask into a structured `ExperienceFrame` (full schema in section 8). This replaces `_detect_intent` and `parse_place_query` as the brain. Extracts: subtype concept (brewery, tapas, omakase, vinyl bar, listening room, gastropub, etc.), vibe, occasion, geo hints, temporal hints, must-haves, soft preferences, negative constraints, ambiguity flags.
- **Inputs.** User prompt, destination, optional anchor (current selected card), trip-day, hotel address (if resolved), prior frame (for refinement carry-over).
- **Outputs.** `ExperienceFrame` (typed Pydantic model).
- **Deterministic vs LLM.** LLM-driven (Sonnet, single call). Constrained output via JSON schema / function-calling. Validators reject malformed frames; on validation failure, fall back to a deterministic frame from `parse_place_query` to avoid total failure.
- **Failure behavior.** On LLM timeout, return a deterministic minimum-viable frame. On JSON validation failure, retry once with a stricter schema, then fall back deterministic.
- **Latency budget.** 600 ms p50, 1200 ms p95. Streamed prefix not used; we need the structured object.
- **Tests.** Frame fixture tests for ~50 representative wife-asks. Each fixture asserts subtype concept, vibe, geo hints, negatives, and `needs_provider_call`. Fuzz tests for adversarial inputs (empty, just punctuation, "asdf", multi-language fragments).
- **Existing code reused.** `parse_place_query` → kept as a deterministic *fallback* only.
- **Existing code deprecated.** `_detect_intent` (`concierge.py:672–701`) and the dispatch table at `concierge.py:270–429` collapse into a frame-driven retrieval planner.

C. Retrieval Planner

- **Purpose.** Take an `ExperienceFrame` and decide *what to do*: pool-only (sufficient cached candidates exist), refine-previous (rerank pool), or fanout-and-fetch (call providers with a planned query set).
- **Inputs.** `ExperienceFrame`, prior pool, prior identity keys, feature flags.
- **Outputs.** `RetrievalPlan` containing: `mode` (`pool_only` | `refine_previous` | `fanout_fetch`), an ordered list of `ProviderQuery` objects (each with deadline, geo hint, type hint), `evidence_calls` (which evidence sources to call, parallel), `top_n_cap`.
- **Deterministic vs LLM.** Deterministic. The plan is composed from frame fields plus simple rules. The LLM does *not* re-think this; the frame already carries the user's intent.
- **Failure behavior.** Always returns at least one `ProviderQuery` (the literal ask + destination) so we cannot accidentally return zero candidates from a planning bug.
- **Latency budget.** < 5 ms (it is a few function calls).
- **Tests.** Plan correctness tests over representative frames. Negative tests: a `more_options` turn produces a `pool_only` plan when the pool has ≥ 3 unused cards.
- **Existing code reused.** Some bits of `_fetch_live_research`'s flag/dispatch logic. Most is rewritten.
- **Existing code deprecated.** The `_FAST_DYNAMIC_INTENTS` set and the binary fast-vs-slow-path decision in `concierge._fetch_live_research`.

D. Provider Query Generator + Executor

- **Purpose.** Execute the `RetrievalPlan`'s provider queries with deadlines and parallelism. Return raw provider results.
- **Inputs.** `RetrievalPlan`.
- **Outputs.** `ProviderResultSet` containing per-query result lists, timings, and per-query failure annotations.
- **Deterministic vs LLM.** Deterministic. Uses asyncio gather with deadlines.
- **Failure behavior.** Per-query timeout. If *all* queries fail (extremely rare), surface a failure that the response builder converts into an honest "we could not reach providers right now; here is what we have from your existing trip" reply.
- **Latency budget.** Total fanout ≤ 1500 ms p95. Each Text Search call ~250–700 ms. Place Details called only after rank, ≤ 400 ms each in parallel.
- **Tests.** Mocked provider tests for parallel fanout, timeout per query, full-fanout failure.
- **Existing code reused.** `google_places.py` Text Search and Place Details clients (kept). `provider_cache.py` for cache layer (kept).
- **Existing code deprecated.** The serial Tavily-extraction-first pattern in `live_research.fetch`. Tavily becomes a parallel optional evidence call only.

E. Verified Place Entity Layer

- **Purpose.** Canonicalize, dedup, and gate provider results into `PlaceEntity` records. This is the trust spine.
- **Inputs.** Raw provider results from D.

- **Outputs.** `List[PlaceEntity]` (deduped, OPERATIONAL, addable). Plus a `RejectedEntities` log for observability (why each rejection happened).
- **Deterministic vs LLM.** Pure deterministic.
- **Failure behavior.** Hard. If a candidate lacks `place_id`, lacks `OPERATIONAL` status, or lacks a Maps URI, it is dropped. No exceptions.
- **Latency budget.** < 50 ms (this is in-memory work).
- **Tests.** Dedup correctness, OPERATIONAL gate, identity-key composition. Regression test for "fake place" — adversarial inputs without place_id are rejected.
- **Existing code reused.** Identity-key logic from `ai.py:95–172`. The `GooglePlaceVerification` model. `_card_passes_trust_gate` from `context_resolver.py`.
- **Existing code deprecated.** Slow-path's ability to mint cards from Tavily articles.

F. Evidence Collector / Fuser

- **Purpose.** Gather and fuse evidence from multiple sources for the top-N candidates after rank: Google `reviewSummary` + `areaSummary` (when available), Place Details fields (rating, count, types, opening hours, photos), Haversine geometry to hotel/anchor, optional Tavily editorial snippets. Tag each evidence item with provenance, confidence, freshness, and which constraint it supports.
- **Inputs.** Top-N `PlaceEntity` records, the `ExperienceFrame`, hotel coords, day/time context.
- **Outputs.** Per-entity `EvidenceBundle` (full schema in section 10).
- **Deterministic vs LLM.** Mostly deterministic (data fetching + structuring). One *optional* small LLM step extracts vibe-relevant snippets from review text when no `reviewSummary` is available; this is a 1-paragraph-in, 3-bullet-out call, batchable across the top-N in one prompt.
- **Failure behavior.** Per-source. If `reviewSummary` is unavailable, fall back to constructing an evidence bundle from `userRatingCount`, top review snippets fetched via Place Details (if API allows), and editorial snippet. If everything fails, flag `evidence_strength=weak` so the reasoner knows.
- **Latency budget.** ≤ 1.2 s p95 (parallel calls). Tavily ≤ 2.0 s with hard drop if late.
- **Tests.** Evidence-bundle correctness; provenance tagging; freshness handling; "weak evidence" path.
- **Existing code reused.** `evidence.py` evidence shape + normalization. Some of `live_research`'s extraction utilities (kept for editorial enrichment).
- **Existing code deprecated.** Tavily-as-substrate pattern.

G. Semantic Fit Ranker

- **Purpose.** Produce the final ordered top-N from candidates + evidence. Feature-based, deterministic, explainable.
- **Inputs.** Candidates, evidence bundles, frame, trip context.
- **Outputs.** Ranked list with per-candidate score breakdown (which features fired, score per feature). Top-N (default 5, max 10).
- **Deterministic vs LLM.** Deterministic. The exact formula is in section 11. The score breakdown is logged for observability and can be surfaced to the user as "why this rank" in a future UI.

- **Failure behavior.** None — this is pure math.
- **Latency budget.** < 30 ms.
- **Tests.** Fixture tests where a brewery scores higher than a generic bar for a brewery ask; a quiet small-plates room scores higher than a cocktail bar for "romantic tapas not too loud"; a chain restaurant gets penalty when "I do not want a chain" is in the frame; popularity loses to subtype-fit + vibe-fit + geo-fit.
- **Existing code reused.** Some of `_category_score` becomes one feature among many.
- **Existing code deprecated.** `_category_score < 0.2` as a *gate* (it stays as a *feature*; a too-bad subtype fit penalizes but does not exclude).

H. Reasoning Engine

- **Purpose.** Generate `whyPick` for each top-N card. Single batched LLM call. Grounded in evidence. Concierge tone. Honest about weak evidence.
- **Inputs.** Top-N entities + their `EvidenceBundle`s, the `ExperienceFrame`, optional comparator instruction.
- **Outputs.** Per-card `whyPick` + optional `tradeoff` + per-card `evidence_citations` (which evidence items the reason references). Plus optional batch-level `summary` line.
- **Deterministic vs LLM.** LLM (Sonnet). Single call covers all top-N. Strict output schema. Validators reject reasons that:
- Reference attributes not present in the evidence bundle.
- Use banned filler ("a great choice", "highly rated", "perfect spot", "hidden gem" without evidence).
- Make geographic claims ("waterfront view", "near the river") unsupported by Place Details + areaSummary.
- Make Michelin/award/historical claims unsupported by evidence.
- **Failure behavior.** On validator rejection, retry once with a refined prompt that names the violation. On second failure, fall back to a deterministic non-templated reason that summarizes only verified fields (rating, neighborhood from Place Details, "verify [claim] when booking" honesty wrapper).
- **Latency budget.** ≤ 1.6 s p95 for batched call.
- **Tests.** Per-fixture reason tests. A "waterfront-claim hallucination" red-team test set. Banned-phrase validator tests.
- **Existing code reused.** `whypick_prompt.py` and `reasoning.py` evidence-grounding pieces.
- **Existing code deprecated.** Per-card LLM reason loops. `_build_dynamic_why` template builder.

I. Follow-up Engine

- **Purpose.** Handle conversational refinement against the existing pool. Implements `top_n`, `best_one`, `compare`, `more_options`, `closer`, `less_touristy`, `more_romantic`, `cheaper`, `not_chain`, `replace_X_with_closer`, "what would you pick?", "make the evening more romantic".
- **Inputs.** Pool, frame delta (the new ask), trip context.
- **Outputs.** Either a card subset (top_n / compare), a comparator answer (best_one + reason), an enriched pool subset (rerank-only ops), or a small refill plus reranked subset (when pool is insufficient).

- **Deterministic vs LLM.** Hybrid. Deterministic operations on the pool (rerank, dedup, subset). LLM call only for opinionated answers ("which would you pick?", comparator) or when the rerank rule needs reasoning over evidence ("more romantic" — needs evidence-based reordering).
- **Failure behavior.** If the pool lacks enough cards for the operation, escalate to a single targeted refill provider call respecting prior identity keys.
- **Latency budget.** ≤ 0.8 s p95 for pure pool operations. ≤ 2.0 s p95 for comparator calls.
- **Tests.** Per-operation fixture tests. Pool-eviction tests. Identity-key preservation tests.
- **Existing code reused.** `context_resolver.resolve_refine_previous` (kept and extended). `result_pool.ContinuationResultPool` (kept; will gain Supabase backing in Phase 3).
- **Existing code deprecated.** None major; mostly extension.

J. Result Pool / Cache

- **Purpose.** Persist verified candidates per trip+canonical-query for fast follow-ups. Two layers: provider-result cache (cross-trip, by query+destination) and trip-pool (per-trip, per-canonical-frame).
- **Inputs.** Verified candidates, identity keys, frame summary.
- **Outputs.** Read/write API.
- **Deterministic vs LLM.** Deterministic.
- **Failure behavior.** Cache miss is harmless (provider call happens). Cache stale is detected by TTL.
- **Latency budget.** Read < 5 ms in-memory; < 50 ms Supabase.
- **Tests.** TTL, eviction, identity-key dedup across pool reads.
- **Existing code reused.** `result_pool.py`, `provider_cache.py`.
- **Existing code deprecated.** None now. Phase 3 promotes pool to Supabase.

K. Observability / Logging

- **Purpose.** Emit structured logs and persist analytics rows. Schema-tolerant.
- **Inputs.** Everything.
- **Outputs.** A single structured `concierge.turn` log line + a `concierge_request_log` row.
- **Deterministic vs LLM.** Deterministic.
- **Failure behavior.** Catches `PGRST204` and similar; logs at most one warning per process per missing column; drops the missing field; retries the insert without it. **Never blocks user response.**
- **Latency budget.** Async, fire-and-forget after response is sent. < 100 ms wall time impact.
- **Tests.** Schema-drift tolerance test. `intent_classifier_version` missing → row still inserts.
- **Existing code reused.** `logging.py`. Modified to be schema-tolerant.
- **Existing code deprecated.** Hard-coded full-row insert.

L. Safety / Trust Gates

- **Purpose.** Pre-response validator. Final pass. Confirms every card is OPERATIONAL + has place_id + has Maps URI; every reason passes the evidence-grounding validator; no banned phrases; no hallucinated claims.
- **Inputs.** Final response payload.
- **Outputs.** Validated response, OR a downgraded response (e.g. retry-once-then-deterministic-reason) if a card's reason fails.
- **Deterministic vs LLM.** Deterministic.
- **Failure behavior.** Drops failed reasons (replacing with deterministic fallback), drops failed cards (extremely rare; logged as P1).
- **Latency budget.** < 50 ms.
- **Tests.** Hallucination red-team test set. Fake-place red-team test set.

M. Frontend Card Contract

- **Purpose.** Typed JSON the frontend already understands, additively extended.
- **Inputs.** Validated response from L.
- **Outputs.** JSON over wire.
- **Deterministic vs LLM.** Deterministic.
- **Failure behavior.** Schema mismatch is a backend bug; the contract is sticky.
- **Tests.** Pydantic contract tests + frontend snapshot tests.

7.3 Module-to-existing-code matrix

New module	Existing file(s) reused	Existing code deprecated
A. Turn Interpreter	`concierge/context.py`, `is_more_options_continuation`, `derive_*_hint`	None
B. Frame Extractor	`parse_place_query` (as fallback only)	`concierge.py:_detect_intent`, `_NIGHTLIFE_PAT`, etc.
C. Retrieval Planner	`_fetch_live_research` flag/dispatch parts	`_FAST_DYNAMIC_INTENTS`, fast-vs-slow binary
D. Provider Executor	`services/google_places.py`, `provider_cache.py`	Serial Tavily-first pattern
E. Verified Place Entity Layer	`ai.py:_card_identity_keys`, `GooglePlaceVerification`	Slow-path Tavily card-minting
F. Evidence Fuser	`concierge/evidence.py`, parts of `live_research` extraction	Tavily-as-substrate
G. Ranker	`_category_score` (as one feature)	`_category_score < 0.2` as a gate
H. Reasoning Engine	`whypick_prompt.py`, `reasoning.py`	Per-card LLM loops, `_build_dynamic_why` templates

New module	Existing file(s) reused	Existing code deprecated
I. Follow-up Engine	`context_resolver.py`, `result_pool.py`	None
J. Pool / Cache	`result_pool.py`, `provider_cache.py`	None now (Phase 3 Supabase backing)
K. Observability	`concierge/logging.py` (schema-tolerant rewrite)	Non-tolerant insert
L. Trust Gates	`_card_passes_trust_gate`, identity-keys	None
M. Frontend contract	`contracts.py`, `PlaceRecommendationsView.tsx`	None

---

8. Data contracts

The system is held together by typed objects. Here are the canonical contracts. Pydantic in Python; mirrored on the wire as JSON.

8.1 ExperienceFrame

The single most important new object. The frame is a flexible structure where cuisine and place_type are **features inside the frame, not the brain.**


```
{
  "literal_ask": "best breweries along the waterfront",
  "normalized_ask": "best breweries along the waterfront in chicago",
  "destination": {
    "city": "Chicago",
    "country": "US",
    "lat": 41.8781,
    "lng": -87.6298
  },
  "trip_id": "f1a1d05b-9565-4b1a-971d-5f448c4a0c16",
  "target_day": null,
  "answer_mode": "place_recommendations",
  "follow_up_mode": "new_search",
  "subtype_concepts": [
    {"label": "brewery", "confidence": 0.95},
    {"label": "taproom", "confidence": 0.6}
  ],
  "place_kind_hints": ["bar", "food_and_drink", "establishment"],
  "must_have": [
    {"label": "near_water", "confidence": 0.9, "kind": "geo"}
  ],
  "soft_preferences": [
    {"label": "best_quality", "confidence": 0.8}
  ],
  "negative_constraints": [],
  "vibe": [],
  "occasion": null,
  "party": {"adults": 2, "kids": 0, "couple": true},
  "cuisine_signals": [],
  "activity_signals": ["drink"],
```

```
  "geography_hints": {
    "anchor": "destination_water_axis",
    "named_places": ["waterfront", "riverwalk", "lakefront"],
    "max_distance_km": 3.0
  },
  "temporal_constraints": {
    "time_of_day": null,
    "day_part": null,
    "weekday": null
  },
  "value_signals": {"luxury_for_less": false, "budget": null, "splurge": false},
  "weather_seasonality": null,
  "accessibility_walkability": {"walkable": true, "max_walk_min": null},
  "ambiguity_flags": ["geo_ambiguity_river_vs_lake"],
  "confidence": 0.88,
  "needs_provider_call": true,
  "can_answer_from_prior_pool": false,
  "explanation_of_parse": "User wants breweries (subtype_concept). 'along the waterfront' is a geo must-hav
}
```

8.1.1 More frame examples

"romantic tapas but not too loud"

```
{
  "literal_ask": "romantic tapas but not too loud",
  "subtype_concepts": [{"label": "tapas", "confidence": 0.95}, {"label": "spanish_small_plates", "confidenc
  "place_kind_hints": ["restaurant"],
  "must_have": [],
  "soft_preferences": [
    {"label": "romantic", "confidence": 0.95},
    {"label": "intimate", "confidence": 0.7}
  ],
  "negative_constraints": [{"label": "loud", "confidence": 0.95, "kind": "ambiance"}],
  "vibe": ["romantic", "quiet"],
  "occasion": "date_night",
  "cuisine_signals": ["spanish", "tapas"],
  "geography_hints": {"anchor": "trip_destination", "max_distance_km": null},
  "needs_provider_call": true
}
```

"nice sushi restaurants with a waterfront view"

```
{
  "literal_ask": "nice sushi restaurants with a waterfront view",
  "subtype_concepts": [{"label": "sushi", "confidence": 0.98}, {"label": "japanese", "confidence": 0.6}],
  "place_kind_hints": ["restaurant"],
  "must_have": [{"label": "waterfront_view", "confidence": 0.9, "kind": "view", "verifiability": "weak"}],
  "soft_preferences": [{"label": "nice_upscale", "confidence": 0.6}],
  "negative_constraints": [],
  "geography_hints": {"anchor": "destination_water_axis", "max_distance_km": 3.0},
  "ambiguity_flags": ["view_verifiability_weak"]
}
```

"upscale seafood but not touristy"

```
{
  "literal_ask": "upscale seafood but not touristy",
  "subtype_concepts": [{"label": "seafood", "confidence": 0.95}],
  "place_kind_hints": ["restaurant"],
  "soft_preferences": [{"label": "upscale", "confidence": 0.9}],
```

```
  "negative_constraints": [{"label": "touristy", "confidence": 0.9, "kind": "audience"}],
  "value_signals": {"luxury_for_less": true, "splurge": false}
}
```

"somewhere fun after dinner, not too loud, good for one drink"

```
{
  "literal_ask": "somewhere fun after dinner, not too loud, good for one drink",
  "subtype_concepts": [{"label": "cocktail_bar", "confidence": 0.6}, {"label": "wine_bar", "confidence": 0.
  "place_kind_hints": ["bar"],
  "soft_preferences": [{"label": "fun", "confidence": 0.7}, {"label": "one_drink_capacity", "confidence": 0
  "negative_constraints": [{"label": "loud", "confidence": 0.9}],
  "occasion": "post_dinner",
  "temporal_constraints": {"day_part": "evening_late", "open_after": "21:30"},
  "ambiguity_flags": ["multiple_subtypes"]
}
```

"best brunch near our hotel"

```
{
  "literal_ask": "best brunch near our hotel",
  "subtype_concepts": [{"label": "brunch", "confidence": 0.98}],
  "place_kind_hints": ["restaurant"],
  "must_have": [{"label": "near_hotel", "confidence": 0.95, "kind": "geo", "anchor_id": "TRIP_HOTEL"}],
  "geography_hints": {"anchor": "hotel", "max_distance_km": 1.5},
  "temporal_constraints": {"day_part": "morning", "open_at": "10:00"}
}
```

"more options" (follow-up)

```
{
  "literal_ask": "more options",
  "follow_up_mode": "more_options",
  "answer_mode": "place_recommendations",
  "carry_over_frame_id": "<prior_frame_id>",
  "needs_provider_call": "depends_on_pool"
}
```

"top 3"

```
{
  "literal_ask": "top 3",
  "follow_up_mode": "refine_previous",
  "rerank_rule": "top_n",
  "n": 3,
  "needs_provider_call": false,
  "can_answer_from_prior_pool": true
}
```

"which is more romantic?"

```
{
  "literal_ask": "which is more romantic?",
  "follow_up_mode": "compare",
  "rerank_rule": "more_romantic",
  "needs_provider_call": false,
  "can_answer_from_prior_pool": true
}
```

"closer to the hotel"

```
{
  "literal_ask": "closer to the hotel",
```

```
  "follow_up_mode": "rerank_prior",
  "rerank_rule": "closer_to_hotel",
  "geography_hints": {"anchor": "hotel"},
  "needs_provider_call": false,
  "can_answer_from_prior_pool": true
}
```

"less touristy"

```
{
  "literal_ask": "less touristy",
  "follow_up_mode": "rerank_prior",
  "rerank_rule": "less_touristy",
  "negative_constraints": [{"label": "touristy", "confidence": 1.0}],
  "needs_provider_call": "if_pool_empty_or_top3_fails_threshold"
}
```

8.2 RetrievalPlan

```
{
  "mode": "fanout_fetch",  // pool_only | refine_previous | fanout_fetch
  "provider_queries": [
    {"id": "pq_literal", "engine": "google_text_search", "query_text": "best breweries along the waterfront
    {"id": "pq_geo", "engine": "google_text_search", "query_text": "breweries Chicago Riverwalk", "geo_bias
    {"id": "pq_alt", "engine": "google_text_search", "query_text": "lakefront breweries Chicago", "geo_bias
  ],
  "evidence_calls": [
    {"id": "ec_review_summary", "engine": "google_place_details", "fields": ["reviewSummary", "areaSummary"
    {"id": "ec_editorial", "engine": "tavily", "query": "best breweries Chicago riverwalk lakefront 2025",
  ],
  "top_n_cap": 5,
  "evidence_required_for": ["near_water", "best_quality"],
  "rerank_rules": []
}
```

8.3 PlaceEntity

```
{
  "id": "pe_chicago_goose_island_fulton",
  "provider_ids": {
    "google_place_id": "ChIJxxxx",
    "yelp_id": null,
    "foursquare_id": null
  },
  "name": "Goose Island Fulton Taproom",
  "normalized_name": "goose island fulton taproom",
  "formatted_address": "1800 W Fulton St, Chicago, IL 60612, United States",
  "coords": {"lat": 41.8868, "lng": -87.6691},
  "neighborhood": "West Loop / Near West Side",
  "place_types": ["bar", "restaurant", "establishment", "food", "point_of_interest"],
  "operational_status": "OPERATIONAL",
  "rating": 4.3,
  "user_rating_count": 1284,
  "price_level": 2,
  "website_uri": "https://...",
  "google_maps_uri": "https://maps.google.com/?cid=...",
  "phone": null,
  "current_opening_hours": {...},
  "photos": [{"name": "places/.../photos/...", "width_px": 4000, "height_px": 3000}],
  "semantic_tags": ["brewery", "taproom", "industrial", "casual"],
  "evidence_snippets": [],
```

```
  "source_provenance": {"primary": "google_text_search", "verified_via": "google_text_search"},
  "verification_confidence": 0.97,
  "addability_status": "ADDABLE",
  "uncertainty_flags": [],
  "freshness": {"fetched_at": "2026-05-05T16:00:00Z", "ttl_s": 1800},
  "identity_keys": ["pid:ChIJxxxx", "gmaps:cid_xxxx", "name_addr:goose_island_fulton_taproom|1800_w_fulton_
}
```

8.4 EvidenceBundle (schema in section 10)

8.5 Ranked output

```
{
  "place_entity_id": "pe_chicago_goose_island_fulton",
  "rank": 2,
  "score": 0.812,
  "score_breakdown": {
    "hard_constraint_pass": true,
    "subtype_fit": 0.95,
    "geo_fit": 0.62,
    "vibe_fit": 0.55,
    "trip_fit": 0.7,
    "evidence_strength": 0.7,
    "popularity_z": -0.1,
    "freshness": 1.0,
    "novelty_diversity": 0.8,
    "personalization": 0.0,
    "penalties": {"unsupported_claim": 0.0, "tourist_trap": 0.0, "chain": 0.0}
  },
  "tradeoff_summary": "Brewery-first; not on the river edge but two blocks back."
}
```

8.6 ConciergeTypedResponse (additive over existing)

The wire contract stays the existing discriminated union (`PlaceRecommendationsResponse | TripAdviceResponse | UnsupportedResponse`). Additions are **all optional** for backward compatibility:

- `frame_summary: Optional[str]` — one-sentence "what we understood you to mean"
- `evidence_strength: Optional[Literal["strong","mixed","weak"]]`
- `tradeoff_chips: Optional[List[str]]` — e.g. `["closer to hotel", "more upscale", "less touristy"]`
- per-card `score_breakdown: Optional[Dict]` (debug-only flag, not always emitted)
- per-card `evidence_citations: Optional[List[EvidenceCitation]]`
- per-card `confidence_label: Optional[Literal["verified","limited_evidence","weak_view_claim"]]`

---

9. Provider strategy

The provider layer implements: cheap recall first, expensive enrichment only after rank, parallel where possible, deadlines on every call.

9.1 Google Places Text Search (v1)

- **Use for.** Primary recall. Send the literal user ask + destination context. Generate up to 3 query variants from the frame for fanout.
- **Do not use for.** Final ranking signals (`types` and `rating` are weak features only). Trust gating beyond `OPERATIONAL`.
- **When to call.** Always on `fanout_fetch`. Skipped on `pool_only` and `refine_previous`.
- **Timeout.** 800 ms per call. Hard cap: 1.2 s.
- **Max candidates.** 12 per call (text search returns up to 20).
- **Cache behavior.** Provider cache by `(canonical_query, destination, geo_bias_hash)`. TTL 30 min. The current `provider_cache.py` is reusable.
- **Failure behavior.** Per-call timeout dropped silently; if all fail, the response builder returns a graceful "we hit a provider hiccup" message instead of fabricating.
- **Cost risk.** Low. Text Search is cheap. Three calls per turn is well within budget.
- **Trust level.** High for entity existence and OPERATIONAL status. Low for "this place fits the user's specific concept" — that is the ranker's job.

9.2 Google Places Nearby Search (v1)

- **Use for.** Optional. Used by the planner only when the frame has a strong geo anchor (`hotel`, named landmark, "within 1 km of X") AND the subtype concept maps to a `types` filter Google supports.
- **Timeout.** 800 ms.
- **Cache.** Same as Text Search.
- **Failure.** Optional call; failure is non-blocking.

9.3 Google Place Details (v1)

- **Use for.** Enrichment of the top-N (default 5) **after** the Ranker runs. Fields we pull: `reviewSummary`, `areaSummary`, `currentOpeningHours`, `userRatingCount`, `rating`, `types`, `photos`, `priceLevel`, `editorialSummary` (where available).
- **Do not use for.** Pre-rank enrichment (too expensive for all candidates). Initial verification (Text Search already returns enough for the `OPERATIONAL` gate).
- **When to call.** After Ranker, in **parallel**, for top-N only.
- **Timeout.** 400 ms per call. If a call misses, the entity goes into the response without `reviewSummary`/`areaSummary` and the reasoner is told `evidence_strength=mixed`.
- **Cache.** `place_id`-keyed entity cache. TTL 6 h for static fields, 30 min for `currentOpeningHours`.
- **Failure.** Per-call. Best-effort.
- **Trust.** High.

9.4 Google `reviewSummary` and `areaSummary` (newer Place Details fields)

- **Use for.** Primary evidence for vibe / "what reviewers say" / "what is the neighborhood" / "is this near the water". These are LLM-summarized, citation-tracked content from Google. They are precisely the evidence we need.
- **Trust level.** High. Treat as quote-able evidence.
- **Caveat.** Availability is uneven — not every place has `reviewSummary`. The Evidence Fuser handles absence by falling back to top review snippets where allowed, plus editorial enrichment.

9.5 Yelp / Foursquare

- **Currently used.** Not in the active concierge flow per the inspection.
- **Future use.** Optional. If we add them, the role is **enrichment only**: subtype tags, neighborhood descriptors, value signals. Never as the verification spine.
- **Not in Phase 1.**

9.6 Tavily / Brave / Serper / editorial

- **Use for.** Evidence enrichment ONLY. Single non-blocking parallel call after Ranker. Find one or two editorial mentions ("Time Out Chicago", "Eater Chicago", "Chicago Mag") that support a vibe/quality claim.
- **Do not use for.** Minting addable cards. Verifying entities. Establishing `OPERATIONAL` status.
- **Timeout.** 2.0 s. Hard drop if late.
- **Cache.** Editorial cache by `(query, destination, week_bucket)` (weekly bucket because editorial freshness is on a weekly cadence). TTL 7 days.
- **Failure.** Best-effort. Reasoner adapts to no editorial evidence.

9.7 Internal cached entities

- Provider-result cache (cross-trip).
- Place-entity cache (by `place_id`).
- Trip-pool cache (by `trip_id` + canonical frame summary).

9.8 Optional future vector index

- Phase 5+. If we accumulate enough Google-verified entities per popular destination, we can build a destination-scoped embedding index of `(name + types + reviewSummary)` for cheap pre-fanout candidate generation. **Not Phase 1.**

9.9 Trust matrix

Provider	Verifies place exists	Verifies OPERATIONAL	Provides addable place_id	Provides Maps URI	Provides vibe evidence	Provides geo coords
Google Text Search v1	yes	yes	yes	yes	partial (types)	yes

Provider	Verifies place exists	Verifies OPERATIONAL	Provides addable place_id	Provides Maps URI	Provides vibe evidence	Provides geo coords
Google Place Details v1	yes	yes	yes	yes	yes (reviewSummary)	yes
Google Nearby Search v1	yes	yes	yes	yes	partial	yes
Yelp	yes (their graph)	partial	no (yelp_id, not place_id)	no (yelp URL)	yes	yes
Foursquare	yes (theirs)	partial	no	no	yes	yes
Tavily / Brave / Serper	NO (web articles)	NO	NO	NO	yes (text snippets)	NO
Editorial blogs	NO	NO	NO	NO	yes	NO

The matrix is the architecture. Only Google can mint addable cards. Everything else is evidence.

---

10. Evidence fusion

Evidence fusion is the bridge between "we found a real place" and "we have a defensible reason to recommend it for this specific ask."

10.1 EvidenceBundle schema


```
{
  "place_entity_id": "pe_chicago_goose_island_fulton",
  "items": [
    {
      "id": "ev_review_summary",
      "kind": "review_summary",
      "source": "google_review_summary",
      "snippet": "A favorite among locals for craft beers; cavernous taproom with industrial-chic decor; po
      "confidence": 0.9,
      "freshness_days": 14,
      "supports_constraints": ["best_quality", "brewery_authentic"],
      "usable_in_reason": true
    },
    {
      "id": "ev_area_summary",
      "kind": "area_summary",
      "source": "google_area_summary",
      "snippet": "Fulton Market District is a former meatpacking area, now restaurants and bars; not on the
      "confidence": 0.85,
      "supports_constraints": ["near_water_qualified"],
      "usable_in_reason": true
    },
    {
      "id": "ev_geo",
      "kind": "geographic",
      "source": "computed",
      "snippet": null,
      "data": {"distance_to_riverwalk_m": 920, "distance_to_lakefront_m": 4200, "is_on_water_axis": false,
```

```
      "confidence": 1.0,
      "supports_constraints": ["near_water_partial"],
      "usable_in_reason": true
    },
    {
      "id": "ev_editorial_1",
      "kind": "editorial",
      "source": "tavily_eater_chicago",
      "snippet": "Goose Island Fulton draws a different crowd from the original Lincoln Park taproom — quie
      "confidence": 0.7,
      "freshness_days": 90,
      "supports_constraints": ["best_quality", "less_touristy"],
      "usable_in_reason": true,
      "url": "https://chicago.eater.com/..."
    },
    {
      "id": "ev_negative_loud",
      "kind": "negative_evidence",
      "source": "google_review_summary_inferred",
      "data": {"loudness_signal": "moderate", "from_phrases": ["cavernous", "after-work groups"]},
      "confidence": 0.6,
      "supports_constraints": ["loudness_concern"],
      "usable_in_reason": true,
      "polarity": "negative"
    },
    {
      "id": "ev_uncertainty_view",
      "kind": "uncertainty",
      "data": {"claim": "waterfront_view", "verifiability": "weak", "reason": "no view evidence in any sour
      "supports_constraints": ["waterfront_view_unsupported"],
      "usable_in_reason": true,
      "polarity": "honesty"
    }
  ],
  "evidence_strength": "mixed",
  "weak_constraints": ["waterfront_view"],
  "strong_constraints": ["brewery_authentic", "operational"]
}
```

10.2 Evidence taxonomy

- **Structured evidence.** Pure data: distance, opening hours, price level, rating, user_rating_count, types. Confidence ~1.0. Always usable.
- **Review-summary evidence.** Google `reviewSummary` (LLM-summarized reviews). Confidence ~0.85. Usable subject to grounding rules.
- **Area-summary evidence.** Google `areaSummary` for the place's neighborhood. Confidence ~0.8. Best for geographic context.
- **Editorial evidence.** Tavily / Brave / Serper enrichment of curated travel/food press. Confidence 0.5–0.8 depending on source. Usable as supporting signal, not as primary evidence for a hard claim.
- **Geographic evidence.** Computed: Haversine distance to hotel, distance to named geographic feature (Riverwalk, Lakefront, Bryant Park, Trastevere), walk-min approximation, water-axis intersection. Confidence ~1.0.
- **Trip-context evidence.** What is in the user's itinerary, what they previously selected/rejected. Confidence ~1.0.

- **Negative evidence.** Signals that the place violates a soft constraint: review keywords for "loud", chain detection, tourist-mass z-score. Confidence varies; polarity flagged.
- **Uncertainty evidence.** Explicit "we cannot verify this claim" signals. These are first-class — the reasoner uses them to express honest uncertainty.

10.3 Per-evidence-item fields

Field	Meaning
`id`	stable id within the bundle
`kind`	one of {`structured`, `review_summary`, `area_summary`, `editorial`, `geographic`, `trip_context`, `negative_evidence`, `uncertainty`}
`source`	which provider/computation produced it
`snippet`	textual quote (if any)
`data`	structured payload (if any)
`confidence`	0–1
`freshness_days`	days since the source was indexed
`supports_constraints`	list of frame-constraint labels this item supports/contradicts
`usable_in_reason`	bool — gates whether the reasoner can quote this
`polarity`	optional: `positive`, `negative`, `honesty`
`url`	optional source URL for citations

10.4 How to prevent hallucinated claims

- The reasoner is given **only** `usable_in_reason: true` **items** as the evidence pool.
- The reasoner output schema requires `evidence_citations: List[evidence_id]` for every assertion.
- The validator (Section 16) cross-checks every reason claim against cited evidence; un-cited claims that touch verifiable attributes (geo, view, hours, price, awards) are rejected.
- Banned phrases without evidence (e.g. "waterfront view") are rejected unless an evidence item with `supports_constraints: ["waterfront_view"]` and `polarity: positive` is cited.

10.5 How to handle weak evidence

When `evidence_strength: weak` for a constraint:

- **"near the waterfront"** with strong geo evidence → say "two blocks from Riverwalk; the room itself does not face the water but the post-dinner walk takes you there."
- **"waterfront view"** with no view evidence → say "I cannot verify a view of the water from inside; verify when booking."
- **"romantic"** with no romance evidence → demote in rank rather than asserting; if the user explicitly asked for romantic and we have no signal, return `confidence_label: limited_evidence` and an honest reason

about what we *can* see ("dinner-room reads quiet, dim lighting in photos, but I do not have specific romance call-outs in the reviews").

10.6 How to handle "not touristy"

- **Quantitative.** Tourist-mass z-score = `(this_place.rating_count - destination.median_rating_count) / destination.std_rating_count`. Z > +1.5 within destination = touristy signal.
- **Categorical.** Chain detection (deterministic match against a known-chain list, e.g. ["The Cheesecake Factory", "Hard Rock Cafe", "Bubba Gump", ...]) — chains lose points hard.
- **Textual.** Review-summary evidence containing "tourist", "tour buses", "lines around the block in summer" — captured as negative evidence.
- **Editorial.** "locals love", "neighborhood favorite", "off the tourist track" → positive `less_touristy` evidence.
- The reasoner uses this as `"a less-trafficked option than the lakefront stretch"` instead of asserting "not touristy" without grounding.

10.7 How to handle "romantic"

- Photo evidence (low light, table-for-two compositions) — limited use, photos are not LLM-readable cheaply.
- Review-summary evidence containing romantic/dinner-date/anniversary tokens.
- Subtype proxies: tasting menu, omakase, wine bar, small-plates with limited seating, candlelight call-outs.
- Editorial evidence calling the place a "date-night staple."
- Combined into a romance score 0–1; reasoner can quote only when the score crosses a threshold.

10.8 How to handle "luxury for less"

Three components:

- **Boutique-feel score:** review keywords like "intimate", "thoughtful", "destination room", "elevated", "craft" minus "loud", "chain", "corporate".
- **Value score:** `price_level` of 2–3 with strong rating + sentiment about value.
- **Anti-touristy score** (above).

Combined into a `luxury_for_less_score`. When the frame has `value_signals.luxury_for_less: true`, this score becomes a soft ranking factor. The reasoner expresses it as "upscale-feeling without the splurge" only when the score crosses a threshold.

10.9 How to handle "good value"

Direct: rating × sentiment-on-value-mentions / price_level. Bounded. Loaded into the ranker as a soft factor when `value_signals` flagged.

10.10 How to handle "open late"

- Hard signal from `currentOpeningHours.regularOpeningHours` if present in Place Details.

- Frame's `temporal_constraints.open_after` becomes a hard filter.
- Reasoner can say "open until 2 AM Friday/Saturday" only when verified from `currentOpeningHours`.

10.11 How to handle "walkable"

- Geographic evidence: distance from hotel/anchor in walk-minutes (via straight-line distance approximation × 1.3 traffic factor).
- Frame's `accessibility_walkability.max_walk_min` becomes a soft preference (cuts in as a penalty for distances above the threshold).

---

11. Ranking model

The Semantic Fit Ranker is deterministic, feature-based, and explainable. The exact formula is open to tuning, but the structure is fixed.

11.1 Score formula

```
score = w_subtype_fit  * subtype_fit
      + w_geo_fit      * geo_fit
      + w_vibe_fit     * vibe_fit
      + w_trip_fit     * trip_fit
      + w_evidence     * evidence_strength
      + w_popularity   * popularity_z_clipped
      + w_freshness    * freshness
      + w_diversity    * diversity_bonus
      + w_personal     * personalization_score
      - p_unsupported  * unsupported_claim_penalty
      - p_tourist      * tourist_trap_penalty
      - p_chain        * chain_penalty
      - p_uncertain    * uncertain_evidence_penalty
      - hard_constraint_violation * 999  // effectively excludes
```

Default Phase 1 weights (subject to tuning by metric review):

```
w_subtype_fit  = 0.30
w_geo_fit      = 0.18
w_vibe_fit     = 0.12
w_trip_fit     = 0.10
w_evidence     = 0.10
w_popularity   = 0.06
w_freshness    = 0.04
w_diversity    = 0.05
w_personal     = 0.05
p_unsupported  = 0.50
p_tourist      = 0.20
p_chain        = 0.15
p_uncertain    = 0.05
```

The dominant positive signal is **subtype fit** (30%). Popularity is 6% — explicitly low so a brewery beats a generic high-rated bar for a brewery ask, and a small-plates room beats a cocktail bar for a tapas ask.

11.2 Feature definitions

- `subtype_fit` ∈ **[0, 1].** How well the candidate matches `frame.subtype_concepts`. Combines:

- Google `types` overlap with frame-derived type hints (e.g. brewery → `bar`, `food`).
- Name-token match against subtype labels ("brewery" in name, "taproom" in name).
- Review-summary token match.
- LLM-derived semantic match (one batched check during evidence fusion: "is this candidate a [subtype]? confidence [0–1]" — done across top-15 in one LLM pass *only when* `types` + name tokens are ambiguous).
- `geo_fit` ∈ **[0, 1].** Inverse-Haversine to anchor (hotel, named landmark, or destination centroid). 1.0 within `max_distance_km`, decays linearly to 0.
- `vibe_fit` ∈ **[0, 1].** Sum of evidence-weighted signals matching `frame.vibe`. Romantic, quiet, lively, casual, upscale.
- `trip_fit` ∈ **[0, 1].** Diversity vs existing itinerary, day-time alignment, party-size fit.
- `evidence_strength` ∈ **[0, 1].** How much usable evidence we have for this candidate. Penalizes "we know nothing about this place" candidates from outranking well-evidenced ones.
- `popularity_z_clipped` ∈ **[-1, 1].** Z-score of `userRatingCount` within destination, clipped. Slight positive but small weight.
- `freshness` ∈ **[0, 1].** 1.0 if all evidence < 90 days old, decays linearly.
- `diversity_bonus` ∈ **[0, 1].** Within the candidate set, penalize duplicates and reward variety (different neighborhoods, different price levels, different subtypes when frame is broad).
- `personalization_score` ∈ **[0, 1].** Per-trip + per-user soft preferences. Phase 4+ feature; default 0 in Phase 1.
- `unsupported_claim_penalty` ∈ **[0, 1].** If the candidate would force the reasoner to make an unsupported claim (e.g. "waterfront view" but no view evidence), penalize so the candidate either ranks lower or is dropped.
- `tourist_trap_penalty` ∈ **[0, 1].** Tourist-mass z-score above threshold + tourist-keyword density.
- `chain_penalty` ∈ **[0, 1].** Chain-detector flag.
- `uncertain_evidence_penalty` ∈ **[0, 1].** Mostly low; activated when evidence is contradictory.

11.3 Tie-breaking rules

When two candidates score within ±0.02:

1. Higher `subtype_fit` wins.
2. Then higher `geo_fit`.
3. Then higher `evidence_strength`.
4. Then more recent freshness.
5. Then `userRatingCount` (mild popularity tiebreak).
6. Then alphabetical for stability.

11.4 Diversity rules

After ranking, a final diversity pass:

- No more than 2 candidates from the same `neighborhood` in top-5.
- No more than 1 candidate from the same `chain` in top-5.
- If subtype is broad, prefer top-5 to span different `subtype_concepts`.

11.5 Worked examples

"best breweries along the waterfront" — Chicago candidates

Candidates after Text Search:

1. Lagunitas Brewing Chicago Taproom (brewery, on the river segment near 18th)
2. Goose Island Fulton (brewery, 12 min walk from Riverwalk East)
3. Goose Island Lincoln Park (brewery, near North Branch but not waterfront)
4. Revolution Brewing Brewpub (brewpub-restaurant, Logan Square — far from water)
5. Fulton Market Kitchen (restaurant, near Fulton — not a brewery, false positive from text)
6. Cruz Blanca (brewery + Mexican food, near West Loop — not waterfront)

Expected ranks under our formula (Phase 1 weights):

1. Lagunitas (subtype 0.95, geo 0.85, vibe 0.6, evidence 0.8) → 0.79
2. Goose Island Fulton (subtype 0.92, geo 0.55, vibe 0.6, evidence 0.7) → 0.66
3. Cruz Blanca (subtype 0.78, geo 0.4, vibe 0.7, evidence 0.6) → 0.55
4. Revolution Brewing (subtype 0.85, geo 0.2, vibe 0.6, evidence 0.7) → 0.50
5. Goose Island Lincoln Park (subtype 0.92, geo 0.15, vibe 0.5, evidence 0.6) → 0.47
6. Fulton Market Kitchen (subtype 0.05, dropped or last) → 0.10 (effectively dropped)

The rank reflects: brewery-first AND on-the-water beats brewery-first AND near-water beats brewery-first AND not-near-water beats not-brewery.

"romantic tapas but not too loud" — Chicago candidates

Generic cocktail bars (current production output) get `subtype_fit ~ 0.3` (cocktail ≠ tapas), even with high popularity. Tapas rooms with quiet evidence get `subtype_fit ~ 0.95` and `vibe_fit ~ 0.85`. The cocktail bars cannot win.

"upscale seafood but not touristy" — touristy seafood near a tourist drag

A high-rating high-rating-count seafood place with tourist-mass z-score > +1.5 gets `popularity_z_clipped ~ +0.5` (tiny gain) but `tourist_trap_penalty ~ 0.7`. Net penalty 0.7 × 0.20 = -0.14. A less-trafficked but well-rated alternative with z-score ~0 ranks higher because the dominant signals (subtype + vibe + evidence) favor it.

11.6 When rating should lose to semantic fit

The whole point. A 4.7-star generic "American" restaurant cannot beat a 4.3-star tapas room when the ask is "tapas." Subtype weight 0.30 vs popularity weight 0.06 enforces this.

11.7 Honest tradeoffs

When a candidate has a positive feature with a tradeoff (e.g. better brewery but farther from water than the top option), the rank still shows it, and the reasoner expresses the tradeoff: "More on the brewery program than the river edge — pick this if the beer matters more than the view."

---

12. Reasoning model

The Reasoning Engine produces `whyPick` per top-N card in a single batched LLM call. It is the part most likely to feel like concierge versus chatbot — and the most likely to hallucinate if not carefully designed.

12.1 Input contract (per turn)

```
{
  "frame_summary": "User wants breweries on/near the waterfront in Chicago. No vibe specified. Wants best."
  "candidates": [
    {
      "id": "pe_lagunitas",
      "name": "Lagunitas Brewing Chicago Taproom",
      "neighborhood": "Pilsen",
      "place_types": ["bar", "food", "restaurant"],
      "rating": 4.4,
      "user_rating_count": 2100,
      "evidence_items": [/* only usable_in_reason: true items */],
      "score_breakdown": {/* from ranker */}
    },
    /* ...up to 5... */
  ],
  "negative_constraints": [],
  "must_have": ["near_water"],
  "frame_ask_focus": ["brewery_authentic", "best_quality", "near_water"]
}
```

12.2 Output contract

```
{
  "cards": [
    {
      "id": "pe_lagunitas",
      "why_pick": "Lagunitas is the only candidate that sits directly on the South Branch — taproom-on-the-
      "tradeoff": "Industrial-feeling room, not romantic — brewery-first.",
      "evidence_citations": ["ev_review_summary", "ev_geo", "ev_area_summary"],
      "confidence_label": "verified",
      "ask_specific_anchors": ["brewery_authentic", "near_water_strong"]
    },
    /* ... */
  ],
  "batch_summary": "All five are brewery-first; ranked by how much of the brewery experience itself meets t
}
```

12.3 Prompt strategy

The system prompt for the Reasoning Engine pins:

- Tone: concierge-friend, not search-engine.
- Length: 1–2 sentences per `why_pick`. Optional 1 sentence per `tradeoff` if the candidate has a notable tradeoff.
- Grounding: every assertion ties to an `evidence_citations` id.
- Honesty: when a frame constraint cannot be fully verified, say so plainly using the standard phrasing (e.g. "verify [claim] when booking").
- Banned: "great choice", "perfect spot", "highly rated", "hidden gem" without evidence, "you'll love it", "trust me."
- Banned: any claim about views, awards, Michelin, opening hours, prices, neighborhoods that cannot be cited.

The user prompt is the input contract serialized.

12.4 Evidence grounding rules

For each assertion in `why_pick`:

- A geographic claim ("on the river", "two blocks from the lakefront") must cite a `geographic` or `area_summary` evidence item.
- A vibe claim ("quiet", "romantic", "industrial") must cite a `review_summary` or `editorial` item that supports it.
- A quality claim ("known for", "called out for") must cite a `review_summary` or `editorial` item.
- A subtype claim ("brewery-first", "tapas-not-cocktail-bar") must cite either `structured` (Google `types` / name) or `review_summary` evidence.
- A negative-honesty claim ("verify the view when booking") must cite an `uncertainty` evidence item (the reasoner is told these are first-class).

12.5 Validators

After the LLM returns:

1. **Schema validator.** Output JSON parses, has all required fields per card.
2. **Citation validator.** Every `evidence_citations` id exists in the input bundle and is `usable_in_reason`.
3. **Banned-phrase regex.** Reject reasons containing the banned superlatives without an evidence anchor.
4. **Geographic-claim validator.** If the reason contains "view", "overlooks", "on the [river|lake|ocean]", "with a view of", check that an evidence item supports the claim.
5. **Award/Michelin/historical validator.** Same rule — reject unless evidence cited.
6. **Length validator.** `why_pick` ≤ 60 words; `tradeoff` ≤ 25 words.
7. **Ask-anchor validator.** Each `why_pick` must reference at least one `frame_ask_focus` concept (verified by an LLM-cheap or regex check on the concept tokens).

12.6 Fallback rules

- If validator rejects a single card's reason, retry that card alone with a stricter "you used unsupported claim X — rewrite without X" prompt.
- If the retry also fails, fall back to a deterministic non-template reason for that card: a 1-sentence summary of verified structured fields ("Tapas restaurant in West Loop; rating 4.5/980 reviews; verify the noise level when booking.") with no vibe claims.
- If batched call times out (>1.6 s p95 + 2 s hard cap), fall back to deterministic per-card reasons for the entire batch and log `reason_source: deterministic_fallback`.

12.7 How to test reason quality

- **Hallucination red-team set.** 50 fixtures where the input forces a temptation to hallucinate (e.g. "best rooftop with skyline views" but no view evidence). The validator must catch every one.
- **Generic-template detector.** A fixture set of 30 outputs from current `_build_dynamic_why` is checked against banned-phrase rules — they must fail. Then a fixture set of new reasoner outputs is checked — they must pass.
- **Ask-anchor coverage.** For each fixture, assert that the produced reason references the frame's `ask_specific_anchors`.
- **Batch latency test.** End-to-end batched call ≤ 1.6 s p95 over 100 trial runs.

12.8 How to avoid serial LLM latency

One batched call. Never N calls for N cards. The batched call covers up to 5 cards in <1.6 s. If the architecture ever adds a separate per-card call, treat it as a regression.

12.9 How to avoid generic text

- The prompt explicitly bans generic phrases.
- The validator catches them.
- The frame's `ask_specific_anchors` are passed in and the validator requires reference to them.
- Manual review fixtures every PR.

12.10 Example reasons (target quality)

"best breweries along the waterfront"

- Lagunitas Brewing Chicago Taproom: "Lagunitas sits directly on the South Branch — taproom-on-the-water, brewery-first program, and the post-pour walk lines up with the Riverwalk you mentioned."
- Goose Island Fulton: "Goose Island Fulton is brewery-first and is twelve minutes' walk from the Riverwalk East entry — so a brewery on the water-axis, not on the water itself."
- Cruz Blanca: "Cruz Blanca is brewery + Mexican food in West Loop, near the water-axis but not on it; pick this if you want food with the beer."

"romantic tapas but not too loud"

- Cira (illustrative): "Cira leans Mediterranean small-plates with a curated wine list; the dinner room reads softer than the Randolph crawl, which fits the 'not too loud' brief better than the cocktail-first rooms."
- Boqueria: "Boqueria is the most explicitly tapas-Spanish on the list with a quieter back room favored for date-night by reviewers — verify the front-room volume on a Friday."

"upscale seafood but not touristy"

- GT Fish & Oyster: "GT is upscale seafood with locals-call-it-out reviews, off the touristy Magnificent Mile drag — closer to a neighborhood-favorite-with-prices-to-match than a tourist destination."
- Joe's Seafood: "Joe's is the popular tourist option; included for completeness but expect tour-bus volume on weekends; pick this only if convenience matters more than locals-feel."

"fun after dinner, not too loud, good for one drink"

- The Aviary: "Aviary is a destination cocktail room that is curated, not loud, and built for one-drink-and-talk; reservation may be required."
- Lost Lake: "Lost Lake is a tiki bar with a fun crowd that stays talk-able most nights; reviewers call out the music staying mid-volume."

"best brunch near our hotel" (hotel = Loop area)

- Lou Mitchell's: "Twelve-minute walk from your hotel; classic Chicago diner brunch; verified open from 5:30 AM."
- Cherry Circle Room: "Eight minutes from your hotel; upscale brunch in a quieter dining room than the Lou Mitchell's queue; fits the 'best' more than the 'fast'."

"less touristy" follow-up: the reasoner re-reasons over the same pool, citing the chain/popularity-z penalty as the basis for the reordering: "Reordered toward locals-favorite signals — pulled X up and Y down because [evidence]."

"which is more romantic?" comparison: the reasoner produces a 1-paragraph judgment: "Cira reads more romantic than Boqueria — the dinner-room evidence is consistent across reviews and the wine list is the focal point, where Boqueria's back room is the romantic part but the front room is high-energy. Pick Cira if the ask is the wine list; Boqueria if the patio matters."

---

13. Follow-up engine

Conversational refinement is where this product earns the "concierge" label. The engine handles a small set of well-defined operations against the prior pool, with a tight latency budget and minimal provider calls.

13.1 Supported operations

Op	Description	Provider call?	LLM call?	Latency
`new_search`	Fresh ask	yes	yes (frame + reason)	p50 4s
`more_options`	Next batch from pool	only if pool dry	no (deterministic rerank), yes only if pool refill	p95 1.2s

Op	Description	Provider call?	LLM call?	Latency
`top_n`	Show top N	no	no	<0.5s
`best_one`	Show single best	no	optional (1 LLM for "why this one")	<0.8s
`compare`	Show 2–N for comparison	no	optional (comparator)	<1.5s
`closer`	Rerank by proximity to anchor	no	no	<0.5s
`less_touristy`	Apply tourist penalty + rerank	no, unless top-3 fail threshold	optional (rewrite reasons)	<1.2s
`more_romantic`	Apply romance feature + rerank	no	optional comparator	<1.5s
`cheaper`	Filter price_level + rerank	no	no	<0.5s
`not_chain`	Apply chain penalty + rerank	no	no	<0.5s
`replace_X_with_closer`	Drop one, fetch closer alternative	yes (1 query)	yes (1 reason)	p95 3s
`make_evening_more_romantic`	Day-plan rewrite around chosen	optional (1 query)	yes (planning)	p95 4s
`what_would_you_pick`	Opinionated single-best with why	no	yes (comparator)	<2s
`replace_X`	Substitute one card	yes (1 query)	yes (1 reason)	p95 3s
`preserve_identity`	Always; no card mutation across follow-ups	n/a	n/a	n/a

13.2 Turn classification

The Turn Interpreter (Module A) classifies the prompt into one of the above. Hybrid approach: regex first, LLM fallback when regex confidence is low.

The current `context.classify_turn` already handles: reset, anchor_new, refine_previous (top_n / best_one / compare / etc.), new_search override, default new_search. We extend with explicit recognition of: `closer`, `less_touristy`, `more_romantic`, `cheaper`, `not_chain`, `replace_X`, `replace_X_with_closer`, `what_would_you_pick`, `make_evening_more_romantic`.

13.3 Prior card pool

- Stored per `(trip_id, canonical_frame_signature)` with TTL (10 min in Phase 1, indefinite in Phase 3 with Supabase).
- Pool entry: list of `PlaceEntity` + `EvidenceBundle` per entity + last rank scores + frame signature.
- Pool reads return raw entities; the Follow-up Engine reranks fresh per follow-up.

13.4 Identity preservation

The pool is the source of truth for card identity. A follow-up never mints a new identity for an existing card. If "more options" merges new candidates with existing ones, identity-key dedup ensures no doublings.

13.5 When provider calls are needed vs skipped

- **Skipped:** `top_n`, `best_one`, `compare`, `closer`, `cheaper`, `not_chain`, `more_romantic` (when evidence in pool suffices), `less_touristy` (when pool top-N has at least 3 below the touristy threshold), `what_would_you_pick`.
- **Required (1 query, bounded):** `replace_X`, `replace_X_with_closer`, `make_evening_more_romantic`, `more_options` when pool < 3 unused.
- **Refill (1–2 variant queries):** `more_options` when pool exhausted; `less_touristy` when pool top-N all fail.

13.6 Avoiding loss of original intent

The frame's `carry_over_frame_id` mechanism preserves the original intent across follow-ups. Each follow-up frame inherits the prior frame's `subtype_concepts`, `must_have`, `negative_constraints`, `vibe`, `geography_hints` unless explicitly overridden by the new prompt. "more options" carries everything; "more options closer to hotel" inherits everything plus adds a `closer` rerank rule.

13.7 Handling specific phrases

- **"more like the second one"** → frame inherits prior frame, anchors on `pool[1]`'s subtype/vibe signature, executes `more_options` with anchor weighting.
- **"closer"** with no anchor named → defaults to hotel (if known), then to destination centroid. Logs the assumption for the user.
- **"less touristy"** → applies tourist penalty doubling temporarily; reranks pool; if top-3 still fail threshold, runs one refill with `less_touristy_seed` (e.g. "neighborhood favorite [subtype] [destination]").
- **"top 3"** → returns pool[0:3] sorted by current rank, no other change.
- **"which would you pick?"** → comparator LLM call over pool[0:3]; returns single recommendation + per-other tradeoff.

---

14. Latency model

Latency is a first-class invariant. Phase 1 targets are aggressive but achievable.

14.1 Stage budgets (p95)

Stage	Budget
Turn Interpreter	100 ms
Frame Extractor (LLM #1)	1200 ms
Retrieval Planner	5 ms
Provider Fanout (parallel Text Search ×3)	1000 ms

Stage	Budget
Place Entity Layer (dedup, gate)	50 ms
Provider Enrichment (Place Details ×top-N parallel)	500 ms
Editorial enrichment (Tavily, parallel, best-effort)	2000 ms (dropped if late)
Evidence Fusion	200 ms
Ranker	30 ms
Reasoning Engine (LLM #2 batched)	1600 ms
Trust gate validator	50 ms
Response build + log	100 ms
Total p95 (without best-effort dropouts)	**~5.0 s**
p95 SLO (deadline)	**7.5 s**
Hard timeout	**9.0 s**
p50 target	**4.0 s**

Editorial enrichment runs in parallel with Place Details and reasoning prep; if it lands in time, the reasoner uses it; if not, it drops.

14.2 Concurrency

- Frame extraction is sequential (the planner needs it).
- Provider fanout (Text Search ×3) is parallel.
- After candidates land, Place Details (top-N) + editorial enrichment + Haversine geo math run in parallel.
- Reasoning is sequential after evidence fusion.
- Logging is fire-and-forget post-response.

14.3 Cache strategy

- **Provider cache hit on Text Search** drops fanout to ~50 ms total.
- **Place entity cache hit** (per `place_id`) drops Place Details to ~5 ms per entity.
- **Pool hit** on follow-up drops everything to ~100–500 ms total.

Target cache hit rates: 60% provider cache, 80% entity cache, 70% pool hit on first follow-up.

14.4 Provider caps

- 3 Text Search calls per turn (hard cap 4).
- Place Details only for top-N (default 5).
- Tavily/Brave: 1 per turn, best-effort, hard 2.0 s drop.

14.5 Progressive return strategy

For Phase 2+, consider streaming response: return verified cards (without `whyPick`) on Place Details landing, then patch in `whyPick` when the reasoner returns. The frontend can show "fetching reasons…" placeholder. This is a **Phase 2** option, not Phase 1.

14.6 Background enrichment

Editorial Tavily call is best-effort. If it lands in time, it improves reasons. If not, the reason draws on Google evidence only. The user-facing latency does not wait on Tavily beyond its drop deadline.

14.7 No serial verification

The current slow path's serial Tavily-extract-then-verify-each-candidate pattern is dead. New verification: candidates come from Google Text Search already verified. Place Details enrichment is parallel.

14.8 No serial per-card reason

Current per-card LLM reason loop is dead. Replaced by one batched call.

14.9 Response quality under timeout

- If Place Details times out, return cards with `evidence_strength: weak` and reasons that lean on structured fields only.
- If Reasoning Engine times out, fall back to deterministic non-template reasons + `confidence_label: limited_evidence`.
- If everything times out, return what we have (likely just verified cards from Text Search) with a brief `frame_summary: "We hit a timeout assembling reasons; here are verified options."` This is honest and actionable.

14.10 Instrumentation

Every stage emits a timing key (`stage_ms`). The structured log captures the full timing dict per turn. Alerting fires when p95 of any stage exceeds budget by >50% over 1 hour.

14.11 Exact targets (recap)

- p50: 4.0 s
- p95: 7.5 s
- Hard timeout: 9.0 s
- Max Google calls: 3 Text Search + 5 Place Details = 8
- Max enrichment calls: 1 (Tavily)
- Max LLM calls: 2 (frame + reason batched). 3 with optional comparator.
- Cache TTLs: provider 30 min, entity 6 h (15 min for hours), pool 10 min Phase 1 → indefinite Phase 3
- Degraded gracefully when: editorial late, Place Details fails for some, reasoning times out

---

15. Caching and memory

The system has four cache tiers, each with a clear role.

15.1 Place entity cache

- **Key:** Google `place_id`.
- **Value:** `PlaceEntity` (canonicalized; section 8.3).
- **TTL:** 6 hours for static fields (name, address, types, coords, rating). 15 minutes for `currentOpeningHours`. Entries refreshed when re-fetched on a turn.
- **Storage:** in-memory in Phase 1; Supabase-backed in Phase 3.
- **Invalidation:** TTL-driven. Manual invalidation on POST `/ai/concierge/cache` (existing endpoint preserved).
- **Hit rate target:** 80% across turns within a single trip.

15.2 Provider/result cache

- **Key:** `(canonical_query, destination, geo_bias_hash)`.
- **Value:** Raw provider results + post-canonicalization entity ids.
- **TTL:** 30 minutes (current setting, kept).
- **Storage:** in-memory Phase 1.
- **Hit rate target:** 60% within a single trip session.

15.3 Trip-pool cache

- **Key:** `(trip_id, canonical_frame_signature)`.
- **Value:** Ranked entities + their evidence bundles + frame signature.
- **TTL:** 10 minutes Phase 1 (in-memory). Supabase-backed in Phase 3 with 24 h TTL or per-trip persistence.
- **Hit rate target:** 70% on first follow-up.

15.4 User preference / session memory

- **Per-trip memory:** trip-level soft preferences accumulated from selected/rejected cards. Stored in Supabase `concierge_trip_preferences` (Phase 4 SQL). In Phase 1, kept in-memory per process.
- **Per-user memory:** persistent across trips. Accumulated slowly. Phase 4+ SQL.
- **Session memory:** the prior pool + last-N prompts feed into the next turn's frame extraction.

15.5 Stale handling and invalidation

- Place Details `currentOpeningHours` re-fetched on every turn that asks about timing.
- Provider results invalidated when destination changes.
- Pool entries invalidated on `reset_search` and on trip-destination change.

15.6 Dedup across turns

- Identity-key set carried across follow-ups. Identity keys live with the pool entry.

15.7 Privacy and security

- No raw user prompts persisted beyond the existing `concierge_messages` table.
- No PII in cache keys.
- Per-trip preferences are user-scoped; RLS-protected at the Supabase tier (extension of existing trip RLS, no new model).

15.8 Does this need SQL?

Phase 1: NO new SQL. Everything stays in-memory. The existing `concierge_messages` and `concierge_request_log` tables are reused. The migration `004` should be applied to live Supabase to fix the `intent_classifier_version` schema-drift error — this is an operational task, not a schema design task.

Phase 3: YES, minimal SQL. Adds `concierge_trip_pool` to persist pool entries beyond process lifetime. Schema:

```
CREATE TABLE IF NOT EXISTS concierge_trip_pool (
  trip_id uuid NOT NULL,
  frame_signature text NOT NULL,
  entities jsonb NOT NULL,
  rank_scores jsonb NOT NULL,
  identity_keys text[] NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  expires_at timestamptz NOT NULL,
  PRIMARY KEY (trip_id, frame_signature)
);
CREATE INDEX ON concierge_trip_pool (trip_id, expires_at DESC);
```

Phase 4: YES, minimal SQL. Adds `concierge_user_preferences` for per-user soft preferences:

```
CREATE TABLE IF NOT EXISTS concierge_user_preferences (
  user_id uuid NOT NULL,
  preference_key text NOT NULL,
  preference_value jsonb NOT NULL,
  weight real NOT NULL DEFAULT 0.0,
  updated_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, preference_key)
);
```

Both rollback-safe (`CREATE TABLE IF NOT EXISTS`). Both isolated from current behavior (additive). Both have RLS policies mirroring `trips` and `concierge_messages`.

---

16. Safety and trust

Trust gates are the product's spine. They are non-negotiable.

16.1 Hard invariants (cannot be relaxed)

- Every addable card has a Google `place_id`.
- Every addable card has `business_status: "OPERATIONAL"`.
- Every addable card has a resolvable `google_maps_uri`.
- No card is minted from Tavily/Brave/Serper/editorial sources.
- Every reason that makes a verifiable claim cites at least one evidence item.
- Every claim about views, awards, Michelin, opening hours, prices, neighborhoods is either verified-and-cited or marked as "verify when booking" honesty.
- No reused card mutates identity across follow-ups.
- No reused card has its `whyPick` rewritten to make a stronger claim than original evidence supports.

16.2 Pre-response validator

The Trust Gate (Module L) is the last step before response. It runs:

1. Schema validation.
2. Identity-key consistency (no duplicate cards, no mutated identity).
3. Trust-invariant check (each card passes hard invariants above).
4. Reason validator (citation, banned phrases, geographic claims, award claims, length, ask-anchor).
5. Confidence-label sanity (cards with weak evidence are labeled).

16.3 Failure modes and safe fallbacks

Failure	Safe fallback
No verified cards from any provider call	Return `place_recommendations` response with empty `cards`, `frame_summary` honest about why ("we couldn't find waterfront breweries that match — try widening to 'breweries near the river'?"), suggest one rephrase. **Never** fall through to text-only with inferred names.
Weak evidence for a constraint	Return cards with `confidence_label: limited_evidence`, reason uses honesty phrasing.
Provider timeout (Text Search)	If at least one of the 3 fanout queries succeeded, proceed with what we have. If all timed out, surface a brief honest "provider hiccup" response.
Place Details enrichment timeout	Return cards with structured fields only. Reasoner falls back to non-vibe-claims.
Tavily timeout	Drop. Reason from Google evidence only.
Frame extraction LLM timeout	Use deterministic fallback frame from `parse_place_query`-style parser.
Reasoning Engine timeout	Deterministic non-template reasons. `reason_source: deterministic_fallback`.
Validator rejects all reasons	Cards return with deterministic safe reasons; log P1.

Failure	Safe fallback
LLM returns malformed JSON	Retry once; on second failure, deterministic fallback.
Logging schema drift	Logger catches `PGRST204`, drops field, retries insert. Never blocks response.
Ambiguous frame (multiple subtypes, contradictory geo)	Frame's `ambiguity_flags` populated; the reasoner adds a "I read your ask as X — adjust if I got it wrong" preamble; response includes `frame_summary`.
Anchor unavailable (hotel not resolved for "near the hotel")	Substitute destination centroid; log assumption; `frame_summary` mentions "I used the destination center because I do not have your hotel pinned yet."

16.4 Hallucination prevention testing

Permanent fixture set, run nightly:

- 20 fixtures forcing temptation to fabricate views.
- 20 fixtures forcing temptation to fabricate awards/Michelin.
- 20 fixtures forcing temptation to fabricate "not touristy" without evidence.
- 20 fixtures forcing temptation to fabricate openings/prices.

Pass rate: 100%. Any failure blocks merge.

---

17. Observability

Observability is the difference between debugging the brewery test in 5 minutes versus 5 hours.

17.1 What every turn logs

Single structured `concierge.turn` log line, plus a `concierge_request_log` row:

- `request_id`
- `trip_id`
- `user_id` (hashed)
- `prompt` (truncated)
- `frame` (compact: subtype_concepts, must_have, vibe, geo_hints, negative_constraints, ambiguity_flags, confidence)
- `turn_mode` (new_search | more_options | refine_previous | etc.)
- `rerank_rule`
- `retrieval_plan` (provider queries with deadlines)
- `provider_calls`: list of `{engine, query, deadline_ms, latency_ms, status, candidate_count}`
- `verification_counts`: `{returned, operational, addable, deduped}`

- `rejection_reasons`: list of `{candidate, reason}` for the top-rejected
- `rank_scores`: top-N with full score breakdown
- `evidence_used`: per top-N entity, the evidence-item ids cited
- `reason_source`: `llm_batched` | `deterministic_fallback`
- `validator_failures`: list of `{card_id, reason}` if any
- `latency_per_stage`: full timing dict
- `cache_hits`: `{provider: bool, entity_count: int, pool: bool}`
- `final_card_count`
- `confidence_labels`: distribution
- `intent_classifier_version`: `frame_extractor_v1` (replaces `router_v2.1` once Frame Extractor ships)
- `pipeline_version`: `semantic_v1`
- `feature_flags`: which flags were active

17.2 What we keep from current logging

- The existing `concierge_request_log` schema is mostly correct. We extend it (additively) with `frame_summary jsonb`, `retrieval_plan jsonb`, `rank_scores jsonb`, `validator_failures jsonb`. All optional, all tolerant.
- The `concierge.context.turn` and `concierge.request_log.persist_failed` log keys remain.

17.3 Schema-tolerant write

The current writer fails the entire insert when a column is missing (the production bug). New writer:

- Catches `PGRST204` and `PGRST116`.
- Logs a single `concierge.logging.schema_drift_detected` warning per process per missing column.
- Drops the missing field from the insert and retries.
- Persists what it can.
- **Never blocks user response.**

A startup self-check (one-time on app boot) compares declared columns against live schema and logs the diff. This converts the "oh this is broken" moment from production-discovery to deploy-time-discovery.

17.4 Should we add a migration?

Yes. Migration `005_apply_004` is operational: confirm `004_concierge_request_log.sql` is applied. If not applied, apply it. If applied but schema cache stale, refresh. The migration content is already correct.

Future additive migration `006_concierge_request_log_extensions.sql` adds the new optional columns (`frame_summary`, `retrieval_plan`, `rank_scores`, `validator_failures`) — all `jsonb DEFAULT '{}'::jsonb`.

17.5 Should analytics failure ever affect user response?

No. The logger is fire-and-forget. The user response goes out on the wire before the log write completes. If logging fails, the user is unaffected.

17.6 Dashboards

Phase 2 deliverable: a small Grafana / Supabase analytics dashboard showing:

- p50/p95 latency per stage over time.
- Card return rate (cards > 0 / total turns).
- Reason validator failure rate.
- Reason fallback usage rate (`deterministic_fallback` vs `llm_batched`).
- Cache hit rates.
- Provider call counts per turn distribution.

17.7 Alerts

- p95 turn latency > 10 s for >5% of turns over 15 min → page.
- Card return rate < 80% over 1 h → page.
- Validator failure rate > 5% over 1 h → notify.
- Logging schema drift detected on startup → notify.
- Provider error rate > 5% over 5 min → notify.

---

18. Frontend contract

Phase 1 minimizes frontend change. The card shape is preserved. New fields are additive and optional.

18.1 Existing fields (preserved)

Per `frontend/src/components/concierge/PlaceRecommendationsView.tsx`:

- `name`
- `cuisine`
- `category`
- `mapsLink`
- `bookingLink`
- `sourceUrl`
- `supportingDetails.{categoryLabel, metaLine, whyPick}`
- `evidence[]`
- `whyPick` (string or `{text, generationMethod}`)

18.2 New optional fields

- `confidence_label`: `"verified" | "limited_evidence" | "weak_view_claim"` — frontend can render a tiny pill ("Verified place", "Limited evidence", "Verify view at booking"). Phase 2.
- `frame_summary` (response-level): one-sentence "we read your ask as ___" — frontend can render in a subtle banner above the cards. Phase 2.
- `evidence_citations`: `[{ id, source, url? }]` per card — frontend can render a small "source" link icon. Phase 2.
- `tradeoff_chips`: response-level `["closer to hotel", "more upscale", "less touristy"]` — clickable refinement chips. Phase 3.
- `score_breakdown`: per-card debug field — only emitted when `?debug=true` query param. Never shown to users. Phase 1.

18.3 Future labels (Phase 4+)

- "Best overall"
- "Most romantic"
- "Closest"
- "Best value"
- "Most unique"
- "Best fit, but weaker waterfront evidence"
- "Verified place"
- "Limited evidence"
- "Near your hotel"

These are implemented as `card.label` enum (optional), set by the Ranker / Reasoner. Frontend renders the label as a tiny corner pill. Phase 4 deliverable.

18.4 Backwards compatibility

Every Phase 1 change is additive. No required fields are removed. No renames. The frontend continues to work unchanged through Phase 1; users see better cards, faster, with better reasons.

18.5 What we do NOT change in Phase 1

- Card layout
- Card colors
- Add-to-itinerary button
- Map integration
- Compare UI
- Existing `evidence[]` rendering

---

19. Phased implementation plan

Five phases. Each is shippable. Each has clear scope, model-of-record for the implementation, files touched, tests, risks, rollback, and feature flag.

19.1 Phase 0 — Repo confirmation and prep

- **Scope.** Confirm inspection findings, apply migration `004` to live Supabase (fixes the `intent_classifier_version` log noise). Add startup schema-drift check. Make `concierge.logging` schema-tolerant.
- **Model.** Sonnet (small focused PR).
- **Files touched.** `backend/app/concierge/logging.py`, `backend/app/main.py` (startup hook), Supabase migration application (operational).
- **Tests.** Schema-drift tolerance test; startup-check test.
- **Risks.** Low. The change is defensive.
- **Rollback.** Trivial.
- **Feature flag.** None — this is a fix.
- **Merge gate.** `/skills/merge_gate.md`.

19.2 Phase 1 — Semantic Place Intelligence vertical slice

- **Scope.** The new architecture's core vertical slice: Frame Extractor (B), Retrieval Planner (C), Provider Executor (D) wrapping existing Google clients, Verified Place Entity Layer (E), Evidence Fuser (F) lite, Ranker (G) with default weights, Reasoning Engine (H) with batched LLM call and validators, Trust Gate (L). Routes through `routes/ai.py` continue to use the existing typed contract; the new pipeline is gated behind a feature flag.
- **Model.** Sonnet (architecture is settled; Sonnet implements). Opus reviews the Frame Extractor prompt and the Reasoning Engine prompt before merge.
- **Feature flag.** `CONCIERGE_SEMANTIC_PLACE_INTELLIGENCE_V1_ENABLED` (default `False`).
- **Behavior matrix.**
- Flag OFF: existing behavior (fast_dynamic_place_search + fallback to live_research).
- Flag ON: new pipeline runs end-to-end. Brewery, tapas, sushi-with-view, upscale-not-touristy, fun-after-dinner, brunch-near-hotel all return verified addable cards.
- **Files likely touched.**
- New: `backend/app/concierge/frame_extractor.py`
- New: `backend/app/concierge/retrieval_planner.py`
- New: `backend/app/concierge/provider_executor.py`
- New: `backend/app/concierge/place_entity_layer.py`
- New: `backend/app/concierge/evidence_fuser.py`
- New: `backend/app/concierge/ranker.py`

- New: `backend/app/concierge/reasoning_engine.py`
- Modified: `backend/app/services/concierge.py` (gates new pipeline behind flag)
- Modified: `backend/app/concierge/contracts.py` (add new optional response fields)
- Modified: `backend/app/concierge/logging.py` (extend log fields)
- Modified: `backend/app/core/config.py` (new flag)
- Reused as fallback: `backend/app/services/fast_dynamic_place_search.py` and `backend/app/services/live_research.py` (no changes).
- **Tests.**
- Frame fixtures for ~30 representative wife-asks.
- Reasoning fixtures for the 8 example asks in the success criteria.
- Hallucination red-team set (50 fixtures).
- Latency test: each stage's p95 budget enforced.
- `card_return_rate ≥ 95%` over the fixture suite.
- **Risks.**
- LLM cost per turn at the new ceiling (~5k tokens). Mitigated by single batched call.
- Frame extraction misclassifies edge cases. Mitigated by deterministic fallback frame.
- Reasoning validator over-rejects valid reasons. Mitigated by retry + manual fixture review every PR.
- **Rollback.** Set the flag to `False`. Existing behavior resumes. Zero code changes needed.
- **Merge gate.** `/skills/merge_gate.md` + Opus review of the two LLM prompts.

19.3 Phase 2 — Evidence fusion and reason quality

- **Scope.** Promote Evidence Fuser (F) to its full schema (Section 10). Add Tavily parallel best-effort. Add Place Details `reviewSummary` / `areaSummary` consumption. Add the full validator suite. Add `frame_summary`, `confidence_label`, `evidence_citations` to the response. Add Phase 2 dashboard.
- **Model.** Sonnet implementation; Opus review of validators.
- **Feature flag.** `CONCIERGE_EVIDENCE_FUSER_V1_ENABLED` (default `False`, layered on top of Phase 1 flag).
- **Files likely touched.** `evidence_fuser.py`, `reasoning_engine.py`, `validators.py` (new), `contracts.py`, frontend `PlaceRecommendationsView.tsx` (additive only).
- **Tests.** Reason-quality fixtures locked; hallucination red-team locked at 100% pass.
- **Risks.** Tavily flakiness. Mitigated by best-effort + drop deadline.
- **Rollback.** Feature flag.
- **Merge gate.** Standard merge gate + reason-quality review.

19.4 Phase 3 — Conversational refinement and pool persistence

- **Scope.** Promote Follow-up Engine (I) to handle full operation set: `closer`, `less_touristy`, `more_romantic`, `cheaper`, `not_chain`, `replace_X`, `what_would_you_pick`. Move Trip-Pool to Supabase (`concierge_trip_pool` table). Add comparator LLM call.
- **Model.** Sonnet implementation; Opus review of comparator prompt.
- **Feature flag.** `CONCIERGE_FOLLOWUP_V2_ENABLED`.
- **SQL.** YES — `006_concierge_trip_pool.sql` and `007_concierge_request_log_extensions.sql`. Both additive, rollback-safe.
- **Files likely touched.** `follow_up_engine.py` (extends `context_resolver.py`), `result_pool.py` (extends to Supabase), `contracts.py`, `routes/ai.py`.
- **Tests.** Per-operation fixtures. Pool persistence across worker restart.
- **Risks.** Migration drift again — apply migration before flag flip.
- **Rollback.** Feature flag + migration is additive.

19.5 Phase 4 — Personalization and trip-context

- **Scope.** Per-trip and per-user soft preference signals. `concierge_trip_preferences` and `concierge_user_preferences` tables. Selected/rejected card → preference update. Hotel proximity → soft signal. Day-time → soft signal. Day-plan rewrite ("make the evening more romantic").
- **Model.** Sonnet.
- **Feature flag.** `CONCIERGE_PERSONALIZATION_V1_ENABLED`.
- **SQL.** YES — additive, rollback-safe.
- **Tests.** Preference accumulation; ranker incorporation; isolation across users.
- **Risks.** Privacy. Mitigated by RLS mirroring existing models.

19.6 Phase 5 — Differentiation layer ("better than travel sites")

- **Scope.** Proactive suggestions ("you have lunch open Wednesday — want me to draft something?"), tradeoff explanations ("worth the splurge?"), "what would you personally pick?", contextual bundles (rainy-day pivot), fast compare and replace.
- **Model.** Opus for design, Sonnet for impl.
- **Feature flag.** `CONCIERGE_DIFFERENTIATION_V1_ENABLED`.
- **Risks.** Scope creep. Strictly gated.

---

20. PR decomposition

The right size: the first PR is one durable vertical slice that cannot be smaller without leaving the brewery test failing. Subsequent PRs add quality and breadth.

20.1 PR catalog

PR-1: Phase 0 — Logging schema-tolerance + migration apply

- **Title:** `concierge: schema-tolerant request log + apply 004`
- **Goal.** Stop the production log noise from `PGRST204` and apply migration 004 to live Supabase.
- **Model.** Sonnet.
- **New chat.**
- **Files.** `backend/app/concierge/logging.py`, `backend/app/main.py` (startup hook).
- **Tests.** Schema-drift unit test; startup-check test.
- **SQL.** Apply existing `004_concierge_request_log.sql` to live (operational). No new migration.
- **Risk.** Low.
- **Acceptance.** Production logs free of `PGRST204` warnings; startup check logs zero drifts after apply.
- **Not included.** Any new Frame Extractor work.
- **Merge gate.** Yes.

PR-2: Phase 1a — Frame Extractor + Retrieval Planner (behind flag)

- **Title:** `concierge: Frame Extractor + Retrieval Planner v1 (flagged)`
- **Goal.** Land the Frame Extractor LLM call and the Retrieval Planner. Wire them so they are invoked when the new flag is on, but the rest of the pipeline still uses the existing fast_dynamic_place_search downstream. This is a pure additive layer.
- **Model.** Sonnet (impl). Opus (prompt review).
- **New chat.**
- **Files.** `frame_extractor.py`, `retrieval_planner.py`, `contracts.py` (ExperienceFrame, RetrievalPlan models), `services/concierge.py` (flag dispatch), `core/config.py` (flag).
- **Tests.** ~30 frame fixtures; retrieval-plan fixtures; latency p95 < 1.4s for frame extraction.
- **SQL.** No.
- **Risk.** Low (flag-gated).
- **Acceptance.** Flag ON: frame extraction runs and is logged. Downstream still uses fast_dynamic. Frame fixtures pass.
- **Not included.** New Provider Executor, new Ranker, new Reasoning Engine.
- **Merge gate.** Yes + Opus prompt review.

PR-3: Phase 1b — Provider Executor + Place Entity Layer + Ranker

- **Title:** `concierge: provider fanout + entity layer + ranker (flagged)`
- **Goal.** Land the parallel provider fanout, entity dedup/canonicalization layer, and the deterministic ranker. Behind the same Phase 1 flag, but downstream still uses the existing reason-builder.
- **Model.** Sonnet.

- **Files.** `provider_executor.py`, `place_entity_layer.py`, `ranker.py`, `services/concierge.py` (extends flag dispatch).
- **Tests.** Provider fanout under timeouts; identity-key dedup; ranker formula correctness.
- **SQL.** No.
- **Risk.** Medium (concurrency). Mitigated by tests.
- **Acceptance.** Brewery query under flag returns ≥3 verified breweries ranked correctly.
- **Not included.** Reasoning Engine, Evidence Fuser.
- **Merge gate.** Yes.

PR-4: Phase 1c — Reasoning Engine + Trust Gate + Validators

- **Title:** `concierge: reasoning engine + trust gate v1 (flagged)`
- **Goal.** Single batched LLM reason call + validator suite + Trust Gate L.
- **Model.** Sonnet impl, Opus prompt + validator review.
- **Files.** `reasoning_engine.py`, `validators.py`, `services/concierge.py` (final wiring), `concierge/logging.py` (extends log fields).
- **Tests.** Reason fixtures for the 8 success-criteria example asks; hallucination red-team 50-fixture set.
- **SQL.** No.
- **Risk.** Medium. Reason quality is the most subjective. Mitigated by Opus review and explicit banned-phrase tests.
- **Acceptance.** Phase 1 vertical slice fully operational. All success-criteria asks return verified cards with concierge-tone reasons. Latency p95 < 7.5 s.
- **Not included.** Tavily editorial integration.
- **Merge gate.** Yes + Opus review.

PR-5: Phase 2 — Evidence fuser full + Tavily editorial + frontend optional fields

- **Title:** `concierge: evidence fuser v1 + editorial + frontend optional`
- **Goal.** Add Place Details `reviewSummary`/`areaSummary` consumption, Tavily parallel best-effort, full evidence-bundle schema, frontend `frame_summary`/`confidence_label`/`evidence_citations`.
- **Model.** Sonnet.
- **Files.** `evidence_fuser.py`, frontend `PlaceRecommendationsView.tsx`, contracts.
- **Tests.** Evidence-bundle fixtures; Tavily late-drop test; frontend snapshot.
- **SQL.** No.
- **Risk.** Low.
- **Acceptance.** Reasons cite editorial sources when available. Latency p95 unchanged (Tavily is best-effort).
- **Merge gate.** Yes.

PR-6: Phase 2 — Dashboard + alerts

- **Title:** `concierge: observability dashboard + alerts`
- **Goal.** Grafana / Supabase analytics dashboard. Alerts wired.
- **Model.** Sonnet.
- **Files.** Dashboard config; alerting config.
- **SQL.** Migration `007_concierge_request_log_extensions.sql` (additive).
- **Risk.** Low.
- **Acceptance.** Dashboard shows Phase 1 metrics live. Alerts fire in test mode.

PR-7: Phase 3a — Follow-up Engine extension

- **Title:** `concierge: follow-up engine v2 (closer/less_touristy/more_romantic/...)`
- **Goal.** Extend `context_resolver` / Follow-up Engine to support the new operations.
- **Model.** Sonnet.
- **Files.** `follow_up_engine.py` (extends `context_resolver.py`), `routes/ai.py`.
- **Tests.** Per-operation fixtures.
- **SQL.** No.
- **Acceptance.** "closer to the hotel", "less touristy", "more romantic", "cheaper", "not a chain" all work against the pool with sub-second response.

PR-8: Phase 3b — Trip pool Supabase migration

- **Title:** `concierge: trip pool persistence (supabase)`
- **Goal.** Migrate `ContinuationResultPool` to Supabase-backed.
- **Model.** Sonnet.
- **SQL.** YES — `008_concierge_trip_pool.sql`.
- **Files.** `result_pool.py`, migration.
- **Tests.** Pool survives worker restart.
- **Risk.** Medium (migration). Mitigated by additive design + flag.
- **Acceptance.** Pool survives restart; pool hit rate unchanged or improved.

PR-9: Phase 4 — Personalization

- **Title:** `concierge: personalization v1 (per-trip + per-user)`
- **Goal.** Trip + user preference accumulation.
- **Model.** Sonnet.
- **SQL.** YES — `009_concierge_preferences.sql`.
- **Files.** `personalization.py` (new), ranker integration, contracts.

- **Tests.** Preference accumulation; isolation across users.
- **Acceptance.** Selected/rejected cards influence subsequent ranks within a trip.

PR-10: Phase 5 — Differentiation layer

- **Title:** `concierge: differentiation v1 (proactive + tradeoffs + day-plan rewrite)`
- **Goal.** "What would you pick?", "make the evening more romantic", proactive suggestions.
- **Model.** Opus (design + impl review), Sonnet (impl).
- **Risk.** Higher. Strictly gated.

20.2 What NOT to bundle

- Do not combine PR-2 + PR-3 + PR-4. The Reasoning Engine review is delicate; isolating it makes review tractable.
- Do not bundle frontend changes with backend pipeline PRs. Keep frontend changes additive in Phase 2 PR-5.
- Do not introduce Personalization (Phase 4) before Phase 3 follow-up engine ships, because personalization needs the follow-up engine's "selected/rejected" signal.

20.3 What to do if a PR fails review

Apply ISSUE_SEVERITY_ROUTING:

- One failed patch → reclassify.
- Two related patches → escalate to full plumbing analysis or split plan.
- The first sign that Phase 1 is too big a single PR is the cue to split into PR-2/3/4 as above.

---

21. Test strategy

21.1 Test categories

Unit tests

- Frame Extractor: schema validation, fallback behavior, JSON-mode failure handling, ambiguity-flag emission.
- Retrieval Planner: plan correctness for each frame shape; pool-only vs fanout-fetch decision.
- Provider Executor: parallel fanout, per-call timeout, all-fail handling.
- Place Entity Layer: dedup correctness; OPERATIONAL gate; identity-key composition.
- Evidence Fuser: per-source bundle assembly; provenance tagging; weak-evidence flagging.
- Ranker: feature computation; weight-formula correctness; tie-breakers; diversity rules.
- Reasoning Engine: prompt construction; output schema; validator integration; fallback path.
- Validators: each validator (citation, banned-phrase, geographic, award, length, ask-anchor).

- Trust Gate: hard-invariant enforcement; rejection paths.
- Follow-up Engine: each operation; pool eviction; identity preservation.
- Logging: schema-tolerance; startup check.

Contract tests

- `ExperienceFrame` Pydantic shape stable.
- `RetrievalPlan` Pydantic shape stable.
- `PlaceEntity` Pydantic shape stable.
- `EvidenceBundle` Pydantic shape stable.
- `ConciergeTypedResponse` discriminated union stable; new optional fields backwards-compatible.
- Frontend snapshot of `PlaceRecommendationsView` props.

Mocked provider tests

- Google Text Search response variants.
- Google Place Details with/without `reviewSummary`.
- Google Place Details timeout.
- Tavily editorial returning, timing out, returning empty.

Reason validator tests

- Reasons referencing fabricated views → rejected.
- Reasons referencing fabricated awards → rejected.
- Reasons with banned superlatives only → rejected.
- Reasons with proper citations → accepted.

Ranking tests

- Brewery ranks above generic bar for brewery ask.
- Tapas room ranks above cocktail bar for tapas ask.
- Chain restaurant penalized when "not a chain" in frame.
- Touristy seafood penalized when "not touristy" in frame.
- Closer-to-hotel reranks correctly without provider call.

Follow-up tests

- "top 3" → pool[0:3], no provider call.
- "best one" → pool[0], no provider call.
- "compare" → pool[0:2] in compare mode, no provider call.
- "more options" with pool of 8 → pool[0:5] of unused, no provider call.

- "more options" with empty pool → 1 provider call, refill.
- "closer to the hotel" → reranks pool by hotel distance.
- "less touristy" → applies penalty + rerank.
- "which is more romantic?" → comparator LLM call.
- "what would you pick?" → opinionated single-best.

Latency / instrumentation tests

- p95 of each stage under budget on the test fixture set.
- Total p95 < 7.5 s for the canonical 8 example asks.
- Pool-hit follow-up p95 < 1.2 s.
- Refine-previous p95 < 0.8 s.
- Per-stage timing emitted on every turn.

Regression tests for current failures

- "best breweries along the waterfront" → ≥ 3 verified breweries returned with concierge reasons.
- "best breweries" → ≥ 3 verified breweries (the second user-reported failure).
- "tapas bar" → tapas-first results, not cocktail-first (the 2026-05-04 wife test).
- "more options" returns ≥ 3 unique cards from the pool when pool has them (the PR-3 motivating bug).

No-fake-card tests

- Adversarial input: provider returns a candidate with no `place_id` → rejected.
- Adversarial input: candidate with `business_status: "CLOSED_PERMANENTLY"` → rejected.
- Adversarial input: name-only matches → rejected unless `place_id` resolves.
- Adversarial input: Tavily returns a "place" not in Google → not minted as a card.

Schema drift / logging tests

- `concierge_request_log` with missing `intent_classifier_version` column → write succeeds with field dropped, single warning logged.
- Multiple drifts in one row → all dropped, single warning per column.
- Successful writes after drift → no error.
- Startup self-check logs a diff when there is one.

21.2 Specific tests for the success-criteria asks

For each of the following, the test asserts: ≥1 verified card, ≥1 ask-specific anchor in the reason, no banned phrases, latency p95 < 7.5 s.

- "best breweries"

- "best breweries along the waterfront"
- "romantic tapas but not too loud"
- "nice sushi restaurants with a waterfront view"
- "upscale seafood but not touristy"
- "cocktail bars with a view"
- "best brunch near our hotel"
- "somewhere fun after dinner, not too loud, good for one drink"
- "more options" (multi-turn fixture)
- "top 3" (multi-turn fixture)
- "which is more romantic?" (multi-turn fixture)
- "which is closest?" (multi-turn fixture)
- "less touristy" (multi-turn fixture)
- "closer to hotel" (multi-turn fixture)

21.3 Failure-mode tests

- Provider timeout on all 3 Text Search → graceful "provider hiccup" response, no fake cards.
- Place Details timeout for all top-N → cards return with structured-only reasons.
- Google verification unavailable → no cards minted; honest response.
- LLM reason timeout → deterministic fallback reasons applied.
- Frame extraction LLM timeout → deterministic fallback frame.
- All combined timeouts → return what we have; honest copy.

21.4 Test fixture maintenance

- Frame fixtures live in `backend/tests/fixtures/frames/` as one JSON per ask.
- Provider response fixtures in `backend/tests/fixtures/providers/`.
- Reason fixtures (expected outputs) in `backend/tests/fixtures/reasons/` (for inspection, not strict equality — too brittle).
- Validator banned-phrase corpus in `backend/tests/fixtures/banned_phrases.txt`.

21.5 Manual review checklist (per PR)

- 5 random fixture reasons read aloud — does each sound like concierge or template?
- 5 random rank breakdowns reviewed — does the order make sense given the ask?
- 5 random frame extractions reviewed — does the parse capture the user's intent?

This is a required step in `merge_gate.md` for the AI Concierge stack.

22. Acceptance examples

For each of these 12 example conversations, expected behavior is specified.

Example 1 — "best breweries along the waterfront"

- **Frame summary.** brewery subtype + waterfront geo must-have; Chicago.
- **Retrieval plan.** 3 Text Search variants: literal ask + destination, geo-biased "Riverwalk breweries", alt "lakefront breweries Chicago".
- **Cards.** ≥3 verified breweries, top one on the water-axis, others within walking distance, all OPERATIONAL.
- **Reasoning.** Each card cites geographic evidence (distance/walk-min) and brewery-evidence (review or types). No fabricated views.
- **Fallback.** None needed.
- **Follow-up.** "more options" returns next batch from pool.

Example 2 — "best breweries" (no geo constraint)

- **Frame summary.** brewery subtype; destination = trip's current location.
- **Retrieval plan.** 2 Text Search variants: literal + destination, "best breweries [destination] 2025".
- **Cards.** ≥4 verified top-rated breweries, ranked by subtype-fit + popularity_z + freshness.
- **Reasoning.** Each cites brewery-evidence + ratings. No geo claims if not asked.

Example 3 — "romantic tapas but not too loud"

- **Frame summary.** tapas subtype, romantic + quiet vibe, anti-loud negative.
- **Retrieval plan.** 2–3 Text Search variants emphasizing tapas ("tapas restaurants Chicago", "Spanish small plates Chicago", "intimate tapas Chicago").
- **Cards.** Tapas-specific rooms, demoted cocktail bars, demoted high-volume rooms based on review evidence.
- **Reasoning.** Each cites quietness/dinner-room evidence; honest about volume risk on weekend nights.
- **Fallback.** If quietness evidence is weak across all candidates, the response includes "could not verify quietness for any candidate; verify on a Friday night when booking" and presents the best subtype-fit options.

Example 4 — "nice sushi restaurants with a waterfront view"

- **Frame summary.** sushi subtype, waterfront view must-have (verifiability flagged weak), upscale soft.
- **Retrieval plan.** Variants emphasizing sushi + waterfront / view.
- **Cards.** Sushi rooms, ranked by subtype-fit + view-evidence (where present). Cards without view evidence are labeled `weak_view_claim`.

- **Reasoning.** Each card honestly states whether the view is verified. "Limited evidence on the view from inside; verify when booking" appears if the view evidence is weak.

Example 5 — "upscale seafood but not touristy"

- **Frame summary.** seafood subtype, upscale soft, anti-touristy negative.
- **Retrieval plan.** Variants emphasizing seafood + locals-favorite / off-tourist-track.
- **Cards.** Seafood rooms with low tourist-mass z-score and good evidence; touristy options demoted with explicit honest tradeoff.
- **Reasoning.** "Locals call this out; lower review-volume than the tourist drag — closer to a neighborhood favorite."

Example 6 — "cocktail bars with a view"

- **Frame summary.** cocktail bar subtype, view must-have (verifiability variable).
- **Retrieval plan.** "rooftop cocktail bars [destination]", "cocktail bars with a view [destination]".
- **Cards.** Rooftop rooms ranked by view evidence + cocktail-program evidence.
- **Reasoning.** "Roof level with skyline view confirmed by area summary" or "claimed rooftop but I can not verify the view from photos; check before booking."

Example 7 — "best brunch near our hotel"

- **Frame summary.** brunch subtype, near-hotel must-have, hotel resolved from trip context.
- **Retrieval plan.** Geo-biased Text Search anchored at hotel coords; max distance 1.5 km.
- **Cards.** Brunch rooms ranked by distance + brunch-evidence + opening-hours alignment.
- **Reasoning.** Each cites distance ("8 min walk from your hotel") and brunch-specific signals.

Example 8 — "somewhere fun after dinner, not too loud, good for one drink"

- **Frame summary.** lounge/cocktail-bar/wine-bar subtype (multiple, broad), fun + quiet vibe, post-dinner timing.
- **Retrieval plan.** Variants for cocktail bars, listening rooms, wine bars; opening-hours filter for late-evening.
- **Cards.** Late-open rooms with quiet/curated evidence; loud rooms demoted; reasons emphasize one-drink-and-talk fit.

Example 9 — "more options"

- **Turn mode.** more_options.
- **Behavior.** Pool fast path. Returns pool[0:5] of unused (after identity-key dedup). p95 < 1.2 s. No provider call when pool has ≥5 unused.

Example 10 — "top 3"

- **Turn mode.** refine_previous, rerank_rule = top_n, n=3.
- **Behavior.** Returns pool[0:3] under current rank. p95 < 0.5 s. No provider call.

Example 11 — "which is more romantic?"

- **Turn mode.** compare, rerank_rule = more_romantic.
- **Behavior.** Reranks pool by `vibe_fit[romantic]`. Comparator LLM call returns 1 paragraph: "X reads more romantic than Y because [evidence]; pick X if [tradeoff]." p95 < 1.5 s. No provider call.

Example 12 — "closer to the hotel"

- **Turn mode.** rerank_prior, rerank_rule = closer_to_hotel.
- **Behavior.** Reranks pool by Haversine to resolved hotel. p95 < 0.5 s. No provider call.

---

23. Red-team critique

The hostile reviewer asks: where does this design fail?

23.1 Where could this still become a bucket router?

- **The Frame Extractor's `subtype_concepts` field could collapse into a fixed enum.** If we constrain the LLM's output to a closed set of labels, we have re-invented categories. **Mitigation:** `subtype_concepts` is an open-vocabulary string list with confidence. The downstream consumer (Ranker) does not key off a closed enum; it uses string match against Google `types`, name tokens, and review-summary tokens. New concepts work without code change.
- **The Ranker's feature list is fixed.** True. But the *features* are general (subtype_fit is an open-vocabulary similarity, not a bucket lookup). The risk is that we hardcode subtype-specific rules. **Mitigation:** the formula is uniform; subtype-specific rules live in the frame, not the Ranker.

23.2 Where could it hallucinate?

- **Reasoning Engine.** The LLM might assert claims even with citations. **Mitigation:** validators check that the claim semantics match the cited evidence (e.g. "near the river" claim cites a `geographic` item with distance < 1km).
- **Frame Extractor.** Could extract a constraint the user did not say. **Mitigation:** frame includes `confidence` per extracted concept; low-confidence concepts are demoted; the user-facing `frame_summary` lets the user catch a misread.
- **Editorial enrichment.** Tavily might return SEO content saying things that are not true. **Mitigation:** editorial evidence gets confidence ≤ 0.7 and is treated as supporting, not primary; Place Details `reviewSummary` outranks editorial.

23.3 Where could it get slow?

- **Frame Extractor LLM call.** ~600–1200 ms p95. If Sonnet has a slow tail, total turn slips. **Mitigation:** budget headroom; deterministic fallback frame on timeout; cache identical-prompt frames within session.

- **Reasoning Engine.** ~1.6 s p95 for batched call. Same risk. **Mitigation:** budget headroom; deterministic fallback; cache identical-frame-and-candidate-set reasons (rare but possible).
- **Place Details fanout for top-N.** Parallel but each call ~250–700 ms. **Mitigation:** 400 ms deadline per call; partial results acceptable.
- **Tavily.** Already best-effort with 2 s drop deadline.
- **Composite tail.** If multiple stages all hit p99 at once, total can blow past 9 s. **Mitigation:** per-stage deadlines + total deadline + graceful degradation.

23.4 Where could it over-call providers?

- **A frame with very broad subtype could trigger fanout with too many variants. Mitigation:** Retrieval Planner caps at 3 Text Search calls per turn.
- **Pool refill in `more_options` could chain. Mitigation:** existing bounded-refill (≤ 2 variants) preserved; total per turn capped at 4 Text Search calls.
- **Place Details called for all candidates instead of top-N. Mitigation:** explicit "after rank" rule; Place Details only for top-N.

23.5 Where could it return popular but wrong places?

- **Popularity-weighted ranking bias.** Suppressed by design: popularity weight is 0.06 vs subtype weight 0.30. **Mitigation:** ranker test fixtures include cases where high-popularity wrong-subtype must lose.

23.6 Where could it fail on vague human language?

- **"a place" / "somewhere" / "anywhere"** — frame extractor returns low-confidence frames; ambiguity_flags populated. **Mitigation:** the frame_summary includes the parse, the user can refine; we do not interrogate.
- **Multilingual** — Phase 1 assumes English. Out-of-scope languages produce a degraded frame; we still attempt the literal Google query.

23.7 Where could it frustrate the wife specifically?

- **Over-honesty on weak evidence becomes scolding.** "Limited evidence on the view; verify when booking" once is fine. Three times per turn becomes lecturing. **Mitigation:** the validator only adds the honesty wrapper when the frame explicitly requires the unverified attribute; if the user did not ask for the view, do not raise it.
- **"more options" returning the same kind repeatedly. Mitigation:** diversity rule in ranking + identity-key dedup across turns.
- **Reasons that are concise but boring. Mitigation:** Opus-reviewed prompt with concierge tone; manual review fixture set.
- **"closer to the hotel" returning places far from hotel. Mitigation:** geo_fit weight + Haversine math; fixture asserts top-3 are within max-walk-min.

23.8 Where could implementation agents overbuild?

- **Adding more LLM calls.** Each LLM call beyond budget is forbidden. The 3-call ceiling is enforced by the `merge_gate.md` review.
- **Adding new ranking features without weight tuning.** Phase 1 weights are fixed; new features tagged `experimental` and gated.
- **Adding personalization in Phase 1.** Out of scope. Stop.
- **Adding new providers.** Out of scope until Phase 2.

23.9 Where could implementation agents underbuild?

- **Skipping validators "to ship faster".** Validators are blocking. PR cannot merge without all validator tests passing.
- **Not implementing the deterministic fallback path.** Fallback is required, not optional.
- **Treating evidence fusion as future work and shipping reason-on-types.** Phase 1's success criteria require ask-anchored reasons grounded in evidence; types alone is insufficient.

23.10 Where could tests give false confidence?

- **Mocked provider tests** can pass while live Google API responses break things. **Mitigation:** a small live-API smoke test set runs nightly against a real Google key (not in CI to avoid cost; in a scheduled job).
- **Reason fixtures with strict equality** would lock in a specific phrasing. **Mitigation:** fixture comparison is structural (cites required evidence ids) + banned-phrase + ask-anchor presence; not literal string match.
- **Latency tests with cached responses** would pass with 100 ms budgets while live is 5 s. **Mitigation:** latency tests use realistic fixture timing; live perf monitored on the dashboard.

23.11 Revised plan based on red-team

- Add explicit "do not interrogate the user" guardrail in the Frame Extractor prompt.
- Add an explicit "honesty wrappers fire only for explicitly-requested unverified attributes" rule in the Reasoning Engine.
- Add diversity-by-subtype to `more_options` to avoid repetition.
- Add a nightly live-API smoke test set.
- Add a reason-quality manual review checkpoint in the PR template (already in `merge_gate.md`).

---

24. Revised final blueprint

After red-team, this is the final.

24.1 What to build first

PR-1 (logging tolerance + 004 apply), then PR-2/3/4 in sequence implementing the Phase 1 vertical slice behind `CONCIERGE_SEMANTIC_PLACE_INTELLIGENCE_V1_ENABLED=False` until ready, then flag flip after the Phase 1 fixture suite passes and Opus has reviewed prompts.

24.2 What to avoid

- Adding "brewery" to existing keyword maps. Do not.
- Bundling Phase 1 into one PR.
- Touching the frontend before Phase 2 (PR-5).
- Adding Personalization or differentiation in Phase 1.

24.3 What to preserve

- Identity-key logic, Google verification gate, typed response contract, refine_previous card reuse, result pool, provider cache, frontend card shape.

24.4 What to delete/deprecate

- `_detect_intent` after Phase 1 flag flip (kept as deprecated import for one release for safety).
- `_FAST_DYNAMIC_INTENTS` set.
- `_NIGHTLIFE_PAT` containing brewery.
- `_SUBTYPE_KEYWORDS` as the brain (kept as fallback only).
- Per-card LLM reason loops.
- Tavily-as-substrate path.

24.5 Feature flag

`CONCIERGE_SEMANTIC_PLACE_INTELLIGENCE_V1_ENABLED` (default `False`). Layered flags for Phase 2, 3, 4, 5.

24.6 SQL needed?

Phase 0: apply existing `004` to live (operational). Phase 1: NO new SQL. Phase 2: tiny additive `007_concierge_request_log_extensions.sql`. Phase 3: `008_concierge_trip_pool.sql`. Phase 4: `009_concierge_preferences.sql`. All additive, all rollback-safe.

24.7 First implementation PR

PR-1 (Phase 0). Logging schema-tolerance + 004 apply. Small, safe, immediately removes production noise. Unblocks Phase 1 development by giving us clean logs.

24.8 Second implementation PR

PR-2 (Phase 1a). Frame Extractor + Retrieval Planner behind flag. Wires the new frame into the existing pipeline as observation-only at first; downstream pipeline still uses fast_dynamic_place_search. Frame is logged, validated against fixtures, but its output does not yet drive retrieval.

Then PR-3 (Provider Executor + Entity Layer + Ranker) and PR-4 (Reasoning Engine + Trust Gate + Validators) complete Phase 1.

24.9 Merge gate checks

For every Phase 1 PR:

- `merge_gate.md` cheap PR review.
- Reason-quality manual review of 5 random fixtures.
- Latency budgets met for affected stages.
- Hallucination red-team set passing 100%.
- No-fake-card tests passing.
- Schema-tolerance tests passing.
- HANDOFF.md updated.
- README.md updated only if behavior changes user-visible (Phase 1 does not until flag flip).

24.10 What success looks like in production

- Brewery test passes: "best breweries along the waterfront" returns ≥3 verified breweries in <7.5 s with concierge-tone reasons.
- The 8 example asks pass.
- Reason validator failure rate < 5%.
- p50 latency < 4 s.
- Card return rate ≥ 95% on verified-place asks.
- Logging error rate near 0.
- Wife (the qualitative metric) reports the system feels different.

---

25. Exact first Sonnet implementation prompt

The prompt below is the next message to send to Claude Sonnet (new chat). It is scoped to PR-1, the Phase 0 logging-and-migration safety fix. PR-2 (Frame Extractor + Retrieval Planner) is its own subsequent prompt.

```
You are working on the Travel Concierge repo on branch
claude/travel-app-architecture-design-yUTIX (or a fresh branch off main, your
call — but coordinate with the user). This is PR-1 of the AI Concierge
Semantic Place Intelligence rollout. The architecture memo lives at
artifacts/ai_concierge_semantic_place_intelligence.pdf (and the markdown
sources artifacts/ai_concierge_architecture_part*.md). Read Section 19.1
(Phase 0) and Section 17 (Observability) before starting.

Severity classification: Level 1 (focused root-cause fix in a small surface).
Repo workflow: docs/ai/skills/bugfix.md and docs/ai/skills/merge_gate.md.

Goal:
Make backend/app/concierge/logging.py schema-tolerant so production stops
emitting concierge.request_log.persist_failed errors when a column is
missing in the live Supabase concierge_request_log table. Currently the
intent_classifier_version column is declared in
backend/db/migrations/004_concierge_request_log.sql:10 but is missing in
live Supabase, causing a PGRST204 error per turn. The durable fix is two-
parts:
1. Make the writer schema-tolerant: catch PGRST204/PGRST116, drop the
   missing field, retry the insert, and log a single warning per process
```

```
   per missing column.
2. Add a startup self-check in backend/app/main.py that compares the
   columns the writer wants to insert against the live concierge_request_log
   schema and logs a structured concierge.logging.schema_drift_detected
   warning at boot if there is a diff.

Do NOT:
- Re-design the Frame Extractor or any other Phase 1 module. That is PR-2+.
- Touch the frontend.
- Weaken Google verification.
- Mint cards from Tavily.
- Change the typed response contract.
- Apply migration 004 to Supabase from code (operational task; user owns).
- Add new feature flags.
- Skip tests.

Execution principles:
- Smallest safe patch.
- Every changed line traces to the goal above.
- No unrelated refactors.
- Root cause first: the writer must NEVER block user response on logging.

Acceptance criteria:
1. backend/app/concierge/logging.py:
   - persist_concierge_request_log catches postgrest.exceptions.APIError
     for codes PGRST204 and PGRST116.
   - On catch, drops the offending column from the row, retries the insert
     ONCE, then logs a single concierge.logging.schema_drift warning per
     process per (table, column) pair.
   - On any other exception, logs at error level but does not raise to
     the caller.
   - Writer is async and never blocks the user response (call site uses
     asyncio.create_task or equivalent fire-and-forget pattern).
2. backend/app/main.py adds a startup hook that:
   - On app boot, queries information_schema.columns for the
     concierge_request_log table.
   - Compares to the set of columns the writer would insert.
   - Logs concierge.logging.schema_drift_detected with the diff if any.
   - Failure to query the schema is logged but does not block startup.
3. backend/tests/test_concierge_observability.py adds (or extends with):
   - A test where the Supabase client raises PGRST204 — assert the row
     persists without the offending column, assert exactly one warning
     logged, assert no exception raised to the caller.
   - A test where the Supabase client raises PGRST204 twice for two
     different columns — assert both columns dropped, two distinct
     warnings, single insert succeeds.
   - A test where the writer is called repeatedly with the same drift —
     assert the warning is emitted once per process per column.
   - A test for the startup self-check that runs against a mocked
     information_schema response with and without drift.

Tests:
- pytest backend/tests/test_concierge_observability.py must pass.
- All other existing tests must still pass.
- Add no new fixtures unrelated to this PR.

Logging / instrumentation:
- New log keys (use these exact strings):
  - concierge.logging.schema_drift (warning, per missing column)
  - concierge.logging.schema_drift_detected (warning, at startup, with
```

```
    structured diff payload)
  - concierge.request_log.persist_succeeded (info, on success after retry)
- The existing concierge.request_log.persist_failed key is reserved for
  unexpected non-schema-drift errors only.

Docs updates:
- Update docs/ai/HANDOFF.md with a new "Last change" entry summarizing the
  fix, including: problem (PGRST204 noise), fix (schema-tolerant writer +
  startup self-check), behavior matrix (drift columns vs no drift), files
  touched, Supabase SQL: No (operational apply of existing 004 still
  required by ops), rollback strategy.
- Note in HANDOFF that the live Supabase project still needs migration 004
  applied; this PR only stops the noise from breaking analytics writes.
- progress_log.md: append a one-line entry.

Do not:
- Update HANDOFF for Phase 1+ work (that comes with PR-2).
- Touch any frontend file.
- Add UI-visible changes.
- Skip tests in any path.

Stop conditions:
- If the durable fix requires changes outside backend/app/concierge/logging.py
  + backend/app/main.py + backend/tests/test_concierge_observability.py +
  docs/ai/HANDOFF.md + progress_log.md, STOP and reclassify as a split-
  plan PR. Report to the user before proceeding.

Final response format:

Severity classification: Level 1
Root cause/plan:
Files changed:
Tests:
Risks:
Supabase SQL: No (apply of 004 is operational, owned by user)
HANDOFF.md edited: Yes + reason
README.md edited: No + reason

Open the PR with title:
"concierge: schema-tolerant request log + startup schema drift check"

PR body summary:
- Stops PGRST204 noise from blocking concierge_request_log writes.
- Adds startup schema-drift detection.
- Operational follow-up: apply migration 004 to live Supabase (user's
  task; this PR makes the live system tolerant in the meantime).

Stop after opening the PR. Do not propose the next implementation prompt.
```

End of memo.

Severity classification: Level 3 (full plumbing analysis + split plan) Root cause/plan: Replace category-bucket routing with Semantic Place Intelligence — Frame Extractor → Retrieval Planner → Provider Executor → Verified Place Entity Layer → Evidence Fuser → Ranker → Reasoning Engine → Trust Gates. Phase 0 fixes logging schema drift; Phase 1 ships the vertical slice behind a flag; Phase 2–5 add evidence quality, conversational refinement, personalization, differentiation. Files changed: None in this memo. PR-1 will modify `backend/app/concierge/logging.py`, `backend/app/main.py`,

`backend/tests/test_concierge_observability.py`, `docs/ai/HANDOFF.md`, `progress_log.md`. Tests: All test categories specified in Section 21. Risks: Spelled out per phase in Section 19; per module in Section 7; system-wide red-team in Section 23. Supabase SQL: NO new SQL in Phase 0/1. Phase 2 adds tiny additive `007`. Phase 3 adds `008`. Phase 4 adds `009`. All additive and rollback-safe. HANDOFF.md edited: Will be edited in PR-1 (not in this memo PR). README.md edited: Not for this memo. Phase 5 may touch README when public-facing behavior shifts.